

LAS MATEMÁTICAS DE LA INTELIGENCIA ARTIFICIAL



Joaquín López Herraiz
Universidad Complutense de Madrid

Las matemáticas de la inteligencia artificial

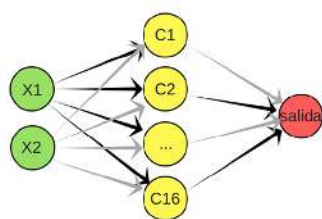
Joaquín L. Herraiz ¹, María Teresa Benavent ², Paula Cordero ¹, Raúl González ¹, Paula Ibáñez ¹,
Juan Antonio Infante ³, Fernando Porqueras ⁴, M. Ángeles Prieto ³, Gema de Jesús Rodríguez².

¹ - Depto. de Estructura de la Materia, Física Térmica y Electrónica. Facultad de Ciencias Físicas. UCM

² - Sec. Dep. de Astronomía y Geodesia. Depto. de Física de la Tierra y Astrofísica. Facultad de Ciencias Matemáticas. UCM

³ - Depto. de Análisis Matemático y Matemática Aplicada. Facultad de Ciencias Matemáticas. UCM

⁴ - Gabinete Informático. Facultad de Ciencias Matemáticas. UCM



Proyecto de Innovación de la Universidad Complutense de Madrid - Convocatoria 2020/2021

Este texto se difunde a través del Proyecto LibreTexts de Recursos Educativos Abiertos (REA) (<https://LibreTexts.org>) y, al igual que los cientos de otros textos disponibles en esta poderosa plataforma, está disponible gratuitamente para leer, imprimir y «consumir». La mayoría de las páginas de la biblioteca, pero no todas, tienen licencias que pueden permitir a las personas hacer cambios, guardar e imprimir este libro. Consulte cuidadosamente las licencias aplicables antes de perseguir dichos efectos.

Los profesores pueden adoptar textos de LibreTexts existentes o remezclarlos para crear rápidamente recursos específicos del curso que satisfagan las necesidades de sus alumnos. A diferencia de los libros de texto tradicionales, los orígenes basados en la web de LibreTexts permiten una poderosa integración de funciones avanzadas y nuevas tecnologías para apoyar el aprendizaje.



La misión de LibreTexts es unir a estudiantes, profesores y académicos en un esfuerzo cooperativo para desarrollar una plataforma en línea fácil de usar para la construcción, personalización y difusión del contenido de REA para reducir la carga de los costos irrazonables de los libros de texto para nuestros estudiantes y la sociedad. El proyecto LibreTexts es una empresa colaborativa multiinstitucional para desarrollar la próxima generación de textos de acceso abierto para mejorar la educación postsecundaria en todos los niveles de educación superior mediante el desarrollo de un entorno de recursos de acceso abierto. El proyecto consiste actualmente en 14 bibliotecas interconectadas y que funcionan de manera independiente y que están siendo optimizadas constantemente por estudiantes, profesores y expertos externos para reemplazar a los libros convencionales en papel. Estas alternativas de libros de texto gratuitos se organizan dentro de un entorno central que está integrado tanto vertical (desde el nivel avanzado hasta el nivel básico) como horizontalmente (en diferentes campos).

Las bibliotecas de LibreTexts son [Desarrollado por MindTouch®](#) y cuentan con el apoyo del Proyecto Piloto de Libros de Texto Abiertos del Departamento de Educación, la Oficina del Rector de UC Davis, la Biblioteca de UC Davis, el Programa de Soluciones de Aprendizaje Asequible de la Universidad Estatal de California y Merlot. Este material se basa en el trabajo apoyado por la Fundación Nacional de Ciencias bajo el número de subvención 1246120, 1525057 y 1413739. A menos que se indique lo contrario, el contenido de LibreTexts [CC POR-NC-SA 3.0](#).

Todas las opiniones, hallazgos y conclusiones o recomendaciones expresadas en este material son de los autores y no reflejan necesariamente los puntos de vista de la Fundación Nacional de Ciencias ni del Departamento de Educación de los EE. UU.

¿Tiene preguntas o comentarios? Para obtener información sobre adopciones o adaptaciones, póngase en contacto con info@LibreTexts.org. Puede encontrar más información sobre nuestras actividades en Facebook (<https://facebook.com/Libretexts>), Twitter (<https://twitter.com/libretexts>) o nuestro blog (<http://Blog.Libretexts.org>).



TABLE OF CONTENTS

1: Portada

- 1.1: Título
- 1.2: Página de Información
- 1.3: Table of Contents
- 1.4: Licensing

2: Introducción a la Inteligencia Artificial y las Redes Neuronales

- 2.1: Introducción General
- 2.2: Perspectiva Histórica
- 2.3: Aplicaciones presentes y futuras
- 2.4: Objetivos del Curso

3: Introducción al Cálculo Multivariante

4: Composición de Funciones. Regla de la Cadena

5: Aproximación Lineal de una Función Escalar de Varias Variables

6: Optimización y Método del Gradiente Descendiente

7: Redes Neuronales

- 7.1: ¿Qué es una red neuronal?
- 7.2: Perceptrón
- 7.3: Funciones de Activación
- 7.4: Funciones de Pérdida
- 7.5: Perceptrón Multicapa
- 7.6: Ejemplo- Regresión lineal

8: Tipos de redes neuronales

- 8.1: Redes neuronales superficiales
- 8.2: Redes neuronales profundas

9: Retropropagación

10: Entrenamiento de una Red Neuronal

11: Apéndices

- 11.1: Índice alfabético
- 11.2: Glosario
- 11.3: Detailed Licensing
- 11.4: Bibliografía Recomendada

1.3: Table of Contents is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

1.4: Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

1.4: Licensing is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

TABLE OF CONTENTS

1: Portada

- 1.1: Título
- 1.2: Página de Información
- Table of Contents
- 1.4: Licensing

2: Introducción a la Inteligencia Artificial y las Redes Neuronales

- 2.1: Introducción General
- 2.2: Perspectiva Histórica
- 2.3: Aplicaciones presentes y futuras
- 2.4: Objetivos del Curso

3: Introducción al Cálculo Multivariante

4: Composición de Funciones. Regla de la Cadena

5: Aproximación Lineal de una Función Escalar de Varias Variables

6: Optimización y Método del Gradiente Descendiente

7: Redes Neuronales

- 7.1: ¿Qué es una red neuronal?
- 7.2: Perceptrón
- 7.3: Funciones de Activación
- 7.4: Funciones de Pérdida
- 7.5: Perceptrón Multicapa
- 7.6: Ejemplo- Regresión lineal

8: Tipos de redes neuronales

- 8.1: Redes neuronales superficiales
- 8.2: Redes neuronales profundas

9: Retropropagación

10: Entrenamiento de una Red Neuronal

11: Apéndices

- 11.1: Índice alfabético
- 11.2: Glosario
- 11.3: Detailed Licensing
- 11.4: Bibliografía Recomendada

CHAPTER OVERVIEW

1: Portada

1.1: Título

1.2: Página de Información

1.3: Table of Contents

1.4: Licensing

1: [Portada](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

Las matemáticas de la inteligencia artificial

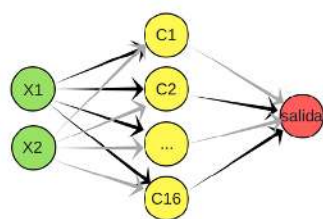
Joaquín L. Herraiz ¹, María Teresa Benavent ², Paula Cordero ¹, Raúl González ¹, Paula Ibáñez ¹,
Juan Antonio Infante ³, Fernando Porqueras ⁴, M. Ángeles Prieto ³, Gema de Jesús Rodríguez².

¹ - Depto. de Estructura de la Materia, Física Térmica y Electrónica. Facultad de Ciencias Físicas. UCM

² - Sec. Dep. de Astronomía y Geodesia. Depto. de Física de la Tierra y Astrofísica. Facultad de Ciencias Matemáticas. UCM

³ - Depto. de Análisis Matemático y Matemática Aplicada. Facultad de Ciencias Matemáticas. UCM

⁴ - Gabinete Informático. Facultad de Ciencias Matemáticas. UCM



Proyecto de Innovación de la Universidad Complutense de Madrid - Convocatoria 2020/2021

Este texto se difunde a través del Proyecto LibreTexts de Recursos Educativos Abiertos (REA) (<https://LibreTexts.org>) y, al igual que los cientos de otros textos disponibles en esta poderosa plataforma, está disponible gratuitamente para leer, imprimir y «consumir». La mayoría de las páginas de la biblioteca, pero no todas, tienen licencias que pueden permitir a las personas hacer cambios, guardar e imprimir este libro. Consulte cuidadosamente las licencias aplicables antes de perseguir dichos efectos.

Los profesores pueden adoptar textos de LibreTexts existentes o remezclarlos para crear rápidamente recursos específicos del curso que satisfagan las necesidades de sus alumnos. A diferencia de los libros de texto tradicionales, los orígenes basados en la web de LibreTexts permiten una poderosa integración de funciones avanzadas y nuevas tecnologías para apoyar el aprendizaje.



La misión de LibreTexts es unir a estudiantes, profesores y académicos en un esfuerzo cooperativo para desarrollar una plataforma en línea fácil de usar para la construcción, personalización y difusión del contenido de REA para reducir la carga de los costos irrazonables de los libros de texto para nuestros estudiantes y la sociedad. El proyecto LibreTexts es una empresa colaborativa multiinstitucional para desarrollar la próxima generación de textos de acceso abierto para mejorar la educación postsecundaria en todos los niveles de educación superior mediante el desarrollo de un entorno de recursos de acceso abierto. El proyecto consiste actualmente en 14 bibliotecas interconectadas y que funcionan de manera independiente y que están siendo optimizadas constantemente por estudiantes, profesores y expertos externos para reemplazar a los libros convencionales en papel. Estas alternativas de libros de texto gratuitos se organizan dentro de un entorno central que está integrado tanto vertical (desde el nivel avanzado hasta el nivel básico) como horizontalmente (en diferentes campos).

Las bibliotecas de LibreTexts son [Desarrollado por MindTouch®](#) y cuentan con el apoyo del Proyecto Piloto de Libros de Texto Abiertos del Departamento de Educación, la Oficina del Rector de UC Davis, la Biblioteca de UC Davis, el Programa de Soluciones de Aprendizaje Asequible de la Universidad Estatal de California y Merlot. Este material se basa en el trabajo apoyado por la Fundación Nacional de Ciencias bajo el número de subvención 1246120, 1525057 y 1413739. A menos que se indique lo contrario, el contenido de LibreTexts [CC POR-NC-SA 3.0](#).

Todas las opiniones, hallazgos y conclusiones o recomendaciones expresadas en este material son de los autores y no reflejan necesariamente los puntos de vista de la Fundación Nacional de Ciencias ni del Departamento de Educación de los EE. UU.

¿Tiene preguntas o comentarios? Para obtener información sobre adopciones o adaptaciones, póngase en contacto con info@LibreTexts.org. Puede encontrar más información sobre nuestras actividades en Facebook (<https://facebook.com/Libretexts>), Twitter (<https://twitter.com/libretexts>) o nuestro blog (<http://Blog.Libretexts.org>).



TABLE OF CONTENTS

1: Portada

- 1.1: Título
- 1.2: Página de Información
- 1.3: Table of Contents
- 1.4: Licensing

2: Introducción a la Inteligencia Artificial y las Redes Neuronales

- 2.1: Introducción General
- 2.2: Perspectiva Histórica
- 2.3: Aplicaciones presentes y futuras
- 2.4: Objetivos del Curso

3: Introducción al Cálculo Multivariante

4: Composición de Funciones. Regla de la Cadena

5: Aproximación Lineal de una Función Escalar de Varias Variables

6: Optimización y Método del Gradiente Descendiente

7: Redes Neuronales

- 7.1: ¿Qué es una red neuronal?
- 7.2: Perceptrón
- 7.3: Funciones de Activación
- 7.4: Funciones de Pérdida
- 7.5: Perceptrón Multicapa
- 7.6: Ejemplo- Regresión lineal

8: Tipos de redes neuronales

- 8.1: Redes neuronales superficiales
- 8.2: Redes neuronales profundas

9: Retropropagación

10: Entrenamiento de una Red Neuronal

11: Apéndices

- 11.1: Índice alfabético
- 11.2: Glosario
- 11.3: Detailed Licensing
- 11.4: Bibliografía Recomendada

1.3: Table of Contents is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

1.4: Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

1.4: Licensing is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

2: Introducción a la Inteligencia Artificial y las Redes Neuronales

1.1. Introducción General

1.2. Perspectiva Histórica

1.3. Aplicaciones presentes y futuras

1.4. Objetivos del Curso

2: Introducción a la Inteligencia Artificial y las Redes Neuronales is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

2.1: Introducción General

This page is a draft and is under active development.

La inteligencia artificial, en especial el aprendizaje automático con redes neuronales, ha experimentado una importante revolución en los últimos tiempos. Esto se ha debido al gran aumento de la cantidad de datos disponibles, el incremento de la capacidad de cálculo de los ordenadores (especialmente con el uso de tarjetas gráficas para cálculo científico), así como importantes mejoras en los algoritmos.

Estos avances están abriendo nuevas posibilidades en muchas áreas, tanto científicas como tecnológicas. Por ejemplo, en el campo de la física de partículas se están empleando para mejorar los análisis de datos en laboratorios como el CERN, también se están aplicando en el análisis de datos astrofísicos, y en el ámbito de la física médica, han demostrado llegar a igualar e incluso superar a expertos en tareas de segmentación y reconocimientos de lesiones en imágenes. También en el campo de la robótica, imitando comportamientos humanos, o en las entrañas de sistemas de recomendación y bots.

Todo esto hace que muchas empresas y proyectos de investigación estén estudiando cómo trasladar y aplicar estas nuevas potentes herramientas a su campo, y haya una importante demanda de expertos en estos temas en el mercado laboral.

A pesar de estos avances, estas materias están solo empezando a formar parte integral de la formación de los estudiantes de física o matemáticas y en general, se circunscriben al ámbito de cursos avanzados y/o de especialización. Sin embargo, el conocer sus fundamentos puede ayudar a los estudiantes a tener una formación más adaptada a las demandas actuales del mercado de trabajo, así como ver aplicaciones prácticas de muchos de los contenidos que se tratan en asignaturas como Cálculo Multivariable y Cálculo Diferencial.

2.1: Introducción General is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

2.2: Perspectiva Histórica

Como sucede en muchas disciplinas, existen diversas interpretaciones sobre el instante en el que se puede determinar el origen de la inteligencia artificial (IA), y las técnicas que se engloban dentro del aprendizaje automático. Esto se debe a que depende en cierta medida de la definición concreta que se use para definir IA o aprendizaje automático, y que se apoya en distintas disciplinas previas como la lógica matemática, la filosofía, la computación, neurociencia... Una lista de sus antecedentes históricos se puede consultar, por ejemplo en [Wikipedia](#).

En cualquier caso, se puede considerar que Inteligencia Artificial fue un concepto principalmente filosófico hasta mediados del siglo XX. En 1950, Alan Turing propuso lo que se conoce como [prueba de Turing](#) una especie de «juego de imitación» en el que participan dos personas y una computadora. Un interrogador plantea preguntas y debe determinar si las respuestas que reciben proceden de una persona o una computadora. Si la computadora es capaz de dar respuestas tan "humanas" que no permitan identificarlas como provenientes de una máquina, según Turing, es inteligente. Éste enfoque ofrece una respuesta sobre la cuestión de cómo estudiar la inteligencia artificial, pero no ofrece las herramientas sobre cómo llegar a crear ese tipo de inteligencia.

En 1956, ya apareció el término "Inteligencia Artificial" en una conferencia en Dartmouth (New Hampshire) y se plantearon unos objetivos muy ambiciosos a corto plazo, que no se cumplieron. Tal como se ha visto posteriormente, la posibilidad de crear este tipo de máquinas inteligentes requiere de una capacidad de cálculo muy lejos de lo que permitían la tecnología de aquella época.

En 1973, la falta de resultados hizo que los gobiernos de EEUU e Inglaterra dejaran de financiar proyectos de IA, y esa falta de financiación se conoce como el "Invierno de la IA". Al comienzo de los años 80, el gobierno de Japón impulsó un ambicioso plan con miles de millones de dólares de inversión en IA, pero a finales de los años 80, de nuevo la falta de los resultados esperados hizo que se volviera a retirar ese tipo de financiación.

Hubo que esperar al siglo XXI para que las técnicas de aprendizaje automático pudieran ser aplicadas con éxito en muchos problemas académicos y de la industria. Los nuevos métodos que se han ido desarrollando en las últimas dos décadas, junto con la capacidad de cómputo y de memoria de las computadoras actuales ha permitido por fin que la IA tenga un gran impacto en la sociedad.

2.2: Perspectiva Histórica is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

2.3: Aplicaciones presentes y futuras

This page is a draft and is under active development.

En la actualidad, el uso de la IA y más concretamente, algoritmos y métodos de aprendizaje automático (incluyendo redes neuronales y aprendizaje profundo) se encuentran muy extendidos, y están siendo usados en la gran mayoría de disciplinas científicas y actividades de la industria.

En muchos casos, todas estas técnicas lo que permiten es realizar un análisis de datos y de la información de un modo mucho más general que lo que se venía usando. Por ejemplo, permiten combinar datos heterogéneos como combinaciones de imágenes y datos numéricos, videos y señales de sensores.. En otros casos, su principal atractivo es eliminar la necesidad de disponer de un modelo teórico previo sobre un fenómeno para poder estudiarlo.

Esto está llevando a un cambio de paradigma en ciencia desde lo que se conoce como "model-driven" a lo que se conoce como "data-driven". Es decir, con la IA se da más importancia a los datos y a lo que se pueda determinar a partir de ellos, que a generar un modelo teórico sencillo y explicable con ellos. Este cambio ofrece importantes ventajas prácticas, pero por supuesto, esto no está exento de polémica y riesgos, y plantea una serie de importantes cuestiones tanto prácticas como filosóficas. Por ejemplo:

- ¿Cómo reconciliar los métodos "clásicos" con los que se obtienen con IA?
- ¿Hasta qué punto debemos seguir usando y desarrollando los métodos previos?
- ¿Cómo lograr "aprender" (en el sentido de generar un modelo que explique un fenómeno y que se pueda formular de manera inteligible para los humanos) de lo que ha "aprendido" una IA?

2.3: Aplicaciones presentes y futuras is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

2.4: Objetivos del Curso

El objetivo de este curso es ofrecer los fundamentos de cálculo matemático en los que se basan las redes neuronales, que han sido la principal herramienta de la IA que ha permitido abarcar problemas complejos.

Existen muchos cursos sobre IA, enfocados principalmente a la programación y el uso de estas nuevas herramientas. También existen algunos cursos en inglés en otras universidades sobre las matemáticas de la IA, que en algunos casos incluyen también contenidos sobre estadística y otras disciplinas que también juegan un importante papel en la IA.

En nuestro caso, nos centraremos en las cuestiones referidas al cálculo, optimización de las redes neuronales, enfocado principalmente a cómo determinar los coeficientes de una red neuronal (lo que se conoce como "entrenamiento de la red neuronal") a partir de una serie de medidas.

2.4: Objetivos del Curso is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

3: Introducción al Cálculo Multivariante

Presentamos inicialmente algunos conceptos como los de función multivariante, derivada parcial, gradiente, así como la composición de funciones de varias variables y la regla de la cadena para su derivación. Todos ellos serán imprescindibles en el desarrollo subsiguiente, en el que se introduce un algoritmo de aprendizaje supervisado muy utilizado para entrenar redes neuronales. Nos valdremos para ello del método del gradiente descendente, en cuya aplicación efectiva se emplea la estrategia de retropropagación, indicada para el cálculo del gradiente de funciones mediante la aplicación recursiva de la regla de la cadena.

Definición 3.1

Definición 1. Una función $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^k$ lleva $\mathbf{x} \in \mathbb{R}^n$ en $\mathbf{y} = \mathbf{f}(\mathbf{x}) \in \mathbb{R}^k$. Se dice que $\mathbb{R}^n$ es el dominio de $\mathbf{f}$ y para cada $\mathbf{x} \in \mathbb{R}^n$, el vector $\mathbf{y} = \mathbf{f}(\mathbf{x})$ es la imagen de $\mathbf{x}$ por $\mathbf{f}$. El conjunto de todas las imágenes se denomina también imagen de $\mathbf{f}$.

- $\mathbf{x} = (x_1, \dots, x_n)^T$.
- $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))^T$ y cada $f_i(\mathbf{x})$ es una función coordenada de $\mathbf{f}$.
- Si $k = 1$, $f(\mathbf{x}) \in \mathbb{R}$.

Observación 3.1

Los vectores $\mathbf{x} \in \mathbb{R}^n$ son vectores columna. Por comodidad, usamos la notación $\mathbf{x} = (x_1, \dots, x_n)^T$ para indicar el traspuesto del vector fila $(x_1, \dots, x_n)$. En ocasiones preferiremos la notación $\mathbf{f}(x, y)$, que resulta más simple que $\mathbf{f}((x, y)^T)$.

Ejemplo 3.1

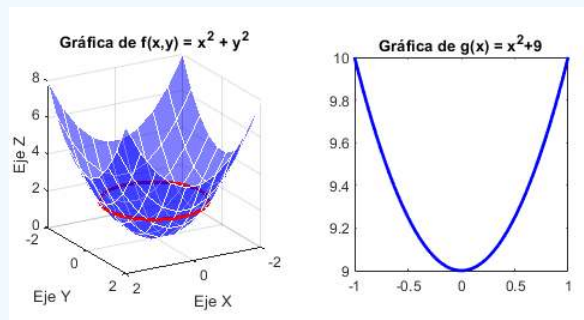
Funciones Lineales

Para números reales fijos a_1, a_2, a_3 , la función lineal f de dominio $\mathbb{R}^3$ dada por $f(x_1, x_2, x_3) = a_1x_1 + a_2x_2 + a_3x_3$ cumple que fijadas dos de las variables.

Por ejemplo $x_1 = -3$ y $x_2 = 5$, la función $g(x_3) = -3a_1 + 5a_2 + a_3x_3$ es una función lineal.

Ejemplo 3.2

Consideramos la función $f(x, y) = x^2 + y^2$, con dominio $\mathbb{R}^2$ e imagen en $\mathbb{R}$. El valor mínimo que toma la función es 0, en el punto $(0, 0)$. En todos los puntos (x, y) que estén sobre la circunferencia centrada en $(0, 0)$ y radio $r > 0$, la función f toma el valor r^2 . Si fijamos una de las variables, por ejemplo $y = 3$, obtenemos la función $g(x) = x^2 + 9$, cuya gráfica es una parábola. Podemos representar la función f , que es la superficie de $\mathbb{R}^3$ de la gráfica siguiente:

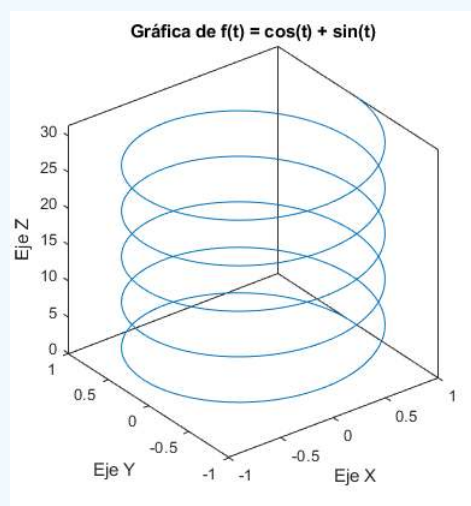


Observación 3.2

Cuando trabajemos con una función $f: \mathbb{R}^n \rightarrow \mathbb{R}$ y fijemos $n - 1$ variables, obtenemos una función con dominio e imagen en $\mathbb{R}$, que podemos analizar con las técnicas ya conocidas.

✓ Ejemplo 3.3

La función $\mathbf{f}: \mathbb{R} \rightarrow \mathbb{R}^2$ dada por $\mathbf{f}(t) = (\cos(t), \sin(t))$ viene definida por las funciones coordenadas $f_1(t) = \cos(t)$ y $f_2(t) = \sin(t)$. La gráfica de $\mathbf{f}$ es una hélice contenida en $\mathbb{R}^3$. La imagen de $\mathbf{f}$ es el conjunto $C := \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\}$, ya que para todo $t \in \mathbb{R}$ se cumple que $\cos^2(t) + \sin^2(t) = 1$.



✓ Ejemplo 3.4

Nos encontraremos con la función sigmoide $\sigma(x) = \frac{1}{1+e^{-x}}$, que es derivable y (¡Compruébalo!) su derivada es $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

Revisamos para una función $f: \mathbb{R} \rightarrow \mathbb{R}$ (es decir: $n = k = 1$), la noción de derivada en un punto x .

✎ Definición 3.2

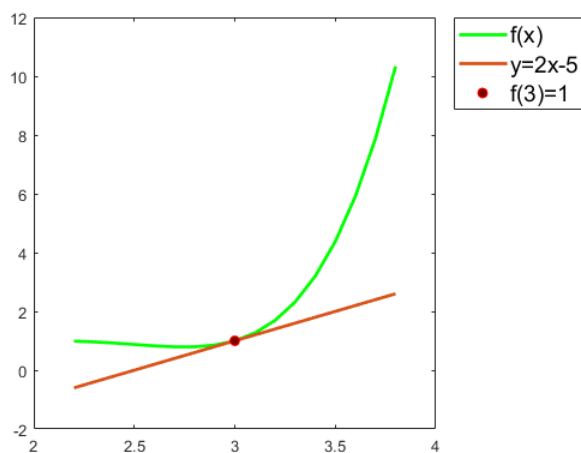
La derivada de f en x , que notamos $f'(x)$, viene dada (¡Cuando existe!) por:

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (3.1)$$

Recuerda que $f'(x)$ es la pendiente de la recta que más se asemeja a la gráfica de la función entre las que pasan por el punto $(x, f(x))$. Como la recta que une los puntos (a, b) y (c, d) tiene pendiente $\frac{d-b}{c-a}$, la recta que une $(x, f(x))$ con $(x+h, f(x+h))$ tiene como pendiente $\frac{f(x+h) - f(x)}{h}$, por lo que $f'(x)$ es el límite de las pendientes de estas rectas.

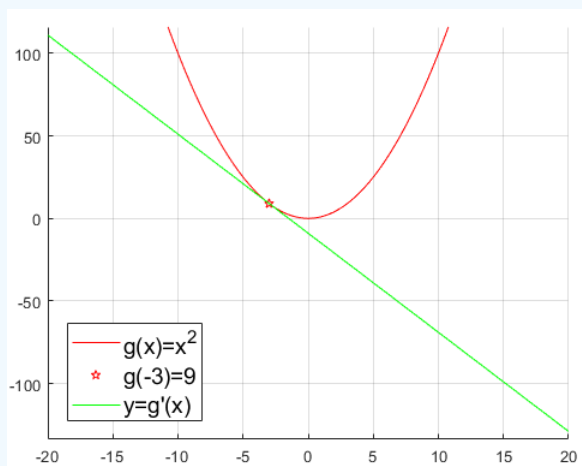
✓ Ejemplo 3.5

Si $f: \mathbb{R} \rightarrow \mathbb{R}$ cumple que $f(3) = 1$ y $f'(3) = 2$, el valor $f(3)$ no puede ser mínimo (ni máximo), ya que la gráfica cerca de 3 es similar a la recta $y = f(3) + f'(3)(x-3) = 1 + 2(x-3)$; es decir, $y = 2x - 5$.



✓ Ejemplo 3.6

Si pensamos en la función $g(x) = x^2$ y $x = -3$, como $g'(-3) = -6$, a la derecha de -3 la función tomará valores menores que $g(-3) = 9$. El valor mínimo de la función, que es $g(0) = 0$, se alcanza en un punto donde la derivada es nula.



Para comprender cómo varía una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ que depende de varias variables empezamos fijando todas las variables salvo una. Consideramos la función resultante (que tiene dominio e imagen en $\mathbb{R}$). Para el ejemplo $f(x, y) = x^2 + y^2$, fijada la variable y obtenemos una función $g_1(x) = x^2 + y^2$, que es decreciente para $x < 0$ y creciente para $x > 0$, ver la figura del ejemplo [Ejemplo 2](#) (se da lo análogo para $g_2(y) = x^2 + y^2$, con x fijo). Nos centramos, por ejemplo, en el punto del plano $(-3, 4)$, para el que $f(-3, 4) = 25$, queremos encontrar valores h_1 y h_2 para los que $f(-3 + h_1, 4 + h_2) < 25$. Podemos conseguirlo siempre $-3 < -3 + h_1 < 0$ y $0 < 4 + h_2 < 4$. Mientras la derivada $g'_1(-3) < 0$ nos alienta a elegir $h_1 > 0$, la derivada $g'_2(4) > 0$ nos impulsa a elegir $h_2 < 0$. La misma estrategia podrá ser utilizada para buscar mínimos de funciones escalares acudiendo a las nociones de derivada parcial y gradiente de la función $f : \mathbb{R}^n \rightarrow \mathbb{R}$. La derivada parcial con respecto a una variable x_j reflejará la tasa de variación de f en la dirección de ese eje.

✎ Definición 3.3

La derivada parcial de la función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ con respecto a la variable x_j en $\mathbf{x} = (x_1, \dots, x_{j-1}, x_j, x_{j+1}, \dots, x_n)^T$, que denotamos $\frac{\partial}{\partial x_j} f(\mathbf{x})$, viene dada por

$$\lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{j-1}, x_j + h, x_{j+1}, \dots, x_n) - f(x_1, \dots, x_{j-1}, x_j, x_{j+1}, \dots, x_n)}{h} \quad (3.2)$$

✓ Ejemplo 3.7

La función $f(x, y) = x^2 + y^2$ tiene como derivadas parciales $\frac{\partial}{\partial x} f(x, y) = 2x$ y $\frac{\partial}{\partial y} f(x, y) = 2y$. La función $f(x_1, x_2, x_3) = a_1 x_1 + a_2 x_2 + a_3 x_3$ tiene como derivada parcial con respecto a x_j el número a_j , para $1 \leq j \leq 3$. La función $f(x, y) = \sin(xy^3) + x^2 y$ tiene como derivadas parciales $\frac{\partial}{\partial x} f(x, y) = y^3 \cdot \cos(xy^3) + 2xy$ y $\frac{\partial}{\partial y} f(x, y) = 3xy^2 \cos(xy^3) + x^2$.

✎ Definición 3.4

Dada una función $f: \mathbb{R}^n \rightarrow \mathbb{R}$ con derivadas parciales $\frac{\partial}{\partial x_j} f(\mathbf{x})$, el gradiente de f se denota por $\nabla f(\mathbf{x})$ y tiene por coordenadas las funciones $\frac{\partial}{\partial x_j} f(\mathbf{x})$, con $1 \leq j \leq n$.

✓ Ejemplo 3.8

La función $f(x, y) = x^2 + y^2$ tiene como gradiente la función $\nabla f(x, y) = (2x, 2y)^T$. La función $f(x_1, x_2, x_3) = a_1 x_1 + a_2 x_2 + a_3 x_3$ tiene como gradiente el vector constante $\nabla f(\mathbf{x}) = (a_1, a_2, a_3)^T$.

Observación 3.3

Como veremos más adelante, el hecho de que la dirección del gradiente $\nabla f(\mathbf{x})$ sea la de máximo crecimiento de la función f en cada punto $\mathbf{x}$ y la dirección dada por su opuesto, $-\nabla f(\mathbf{x})$, la de mayor decrecimiento, permite establecer una estrategia para minimizar la función escalar $f(\mathbf{x})$. Para la función $f(x, y) = x^2 + y^2$ en el punto $(-3, 4)$ hemos elegido valores (h_1, h_2) de signos opuestos al gradiente $(-6, 8)$ para obtener puntos $(-3 + h_1, 4 + h_2)$ en los que f toma valores menores que $f(-3, 4) = 25$.

Si trabajamos con una función $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^k$ de k coordenadas $f_1, f_2, \dots, f_k$, cuando existan las n derivadas parciales de cada una de ellas podemos considerar la función matricial de k filas y n columnas formada por los k gradientes ∇f_k . Se trata de la *matriz jacobiana* de la función $\mathbf{f}$.

3: Introducción al Cálculo Multivariable is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

4: Composición de Funciones. Regla de la Cadena

Si la imagen de una función $\mathbf{f}$ está contenida en el dominio de una función $\mathbf{g}$, puede aplicarse $\mathbf{g}$ a los valores $\mathbf{f}(\mathbf{x})$ y obtener la composición $\mathbf{g} \circ \mathbf{f}$.

Por ejemplo, $\mathbf{f}: \mathbb{R} \rightarrow \mathbb{R}^2$ dada por $\mathbf{f}(t) = (\cos(t), \sin(t))$ puede componerse con la función $g(x, y) = x^2 + y^2$ o con $g(x, y) = \sin(xy^3) + x^2y$, obteniendo, respectivamente, las funciones $g \circ \mathbf{f}: \mathbb{R} \rightarrow \mathbb{R}$ dadas por $(g \circ \mathbf{f})(t) = g(\cos(t), \sin(t)) = \cos^2(t) + \sin^2(t) \equiv 1$ y $(g \circ \mathbf{f})(t) = g(\cos(t), \sin(t)) = \sin(\cos(t) \sin^3(t)) + \cos^2(t) \sin(t)$. Para las funciones $\mathbf{f}(x_1, x_2, x_3) = a_1x_1 + a_2x_2 + a_3x_3$ y $\mathbf{h}(t) = (\cos(t), \sin(t))$ podemos considerar la composición $\mathbf{h} \circ \mathbf{f}: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ que se expresa por

$$(\mathbf{h} \circ \mathbf{f})(x_1, x_2, x_3) = \mathbf{h}(a_1x_1 + a_2x_2 + a_3x_3) = (\cos(a_1x_1 + a_2x_2 + a_3x_3), \sin(a_1x_1 + a_2x_2 + a_3x_3)) \quad (4.1)$$

pero no $\mathbf{f} \circ \mathbf{h}$, ya que la imagen de $\mathbf{h}$ está contenida en $\mathbb{R}^2$ y el dominio de $\mathbf{f}$ es $\mathbb{R}^3$. En general, consideramos composiciones de funciones $\mathbf{g} \circ \mathbf{f}$, donde $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^k$ y $\mathbf{g}: \mathbb{R}^k \rightarrow \mathbb{R}^m$; analizamos las m coordenadas de la composición y, para cada una de ellas, estudiamos las respectivas derivadas parciales con respecto a las n variables. Por ello, en los ejemplos nos centraremos en el caso $m = 1$.

Regla de la Cadena

Dadas dos funciones derivables f y g con dominio e imagen en $\mathbb{R}$ y su composición $g \circ f$, se calcula la derivada de la función $g \circ f$ a través de la denominada regla de la cadena. En concreto, $(g \circ f)'(x) = g'(f(x))f'(x)$.

✓ Ejemplo 4.9

La función $f(x) = x^2$ se compone con la función $g(x) = e^x$ y da lugar a $(g \circ f)(x) = e^{x^2}$, cuya derivada es $(g \circ f)'(x) = e^{x^2} 2x$. Análogamente, $(f \circ g)(x) = (e^x)^2$ (de hecho, $(f \circ g)(x) = e^{2x}$) y por la regla de la cadena $(f \circ g)'(x) = 2e^x e^x = 2e^{2x}$.

Veamos cómo operar en el caso en que las funciones tengan dominio o imagen en $\mathbb{R}^d$, con $d > 1$.

- $\mathbf{f}: \mathbb{R}^2 \rightarrow \mathbb{R}$ y $g: \mathbb{R} \rightarrow \mathbb{R}$. Utilizamos las variables s, t para $\mathbf{f}$ y la variable x para g . Entonces $y = g(x) = (g \circ \mathbf{f})(s, t) := F(s, t)$ y

$$\frac{\partial}{\partial s} F = \frac{d}{dx} g \frac{\partial}{\partial s} f \quad (4.2)$$

Análogamente,

$$\frac{\partial}{\partial t} F = \frac{d}{dx} g \frac{\partial}{\partial t} f \quad (4.3)$$

✓ Ejemplo 4.10

Si $f(s, t) = s(1 - t^3)$ y $g(x) = e^x$, la función $F(s, t) = (g \circ f)(s, t) = g(s(1 - t^3)) = e^{s(1 - t^3)}$ tiene como gradiente $\nabla F(s, t) = (e^{s(1 - t^3)}(1 - t^3), e^{s(1 - t^3)}s(-3t^2))^T$.

- $\mathbf{f} = (f_1, f_2)^T: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ y $g: \mathbb{R}^2 \rightarrow \mathbb{R}$. Utilizamos las variables s, t para $\mathbf{f}$ y las variables x, y para g . Entonces $z = g(x, y) = (g \circ \mathbf{f})(s, t) := F(s, t)$ y

$$\frac{\partial}{\partial s} F = \frac{\partial}{\partial x} g \frac{\partial}{\partial s} f_1 + \frac{\partial}{\partial y} g \frac{\partial}{\partial s} f_2 \quad (4.4)$$

Análogamente,

$$\frac{\partial}{\partial t} F = \frac{\partial}{\partial x} g \frac{\partial}{\partial t} f_1 + \frac{\partial}{\partial y} g \frac{\partial}{\partial t} f_2 \quad (4.5)$$

✓ Ejemplo 4.11

Si $(x, y) = \mathbf{f}(s, t) = (s^2t, s(4 - 3t))$, $g(x, y) = \cos(x - 2y)$ y F es la función $F(s, t) := (g \circ \mathbf{f})(s, t) = g(s^2t, s(4 - 3t)) = \cos(s^2t - 2s(4 - 3t))$ admite derivadas parciales. Calcula directamente la derivada parcial con respecto a cada una de las variables y hazlo usando la regla de la cadena multivariante para ver que coinciden.

- Si $\mathbf{f} = (f_1, \dots, f_k)^T : \mathbb{R}^n \rightarrow \mathbb{R}^k$ y $\mathbf{g} : \mathbb{R}^k \rightarrow \mathbb{R}$ admiten derivadas parciales, las de la función $F = \mathbf{g} \circ \mathbf{f}$ vienen dadas, para cada $1 \leq j \leq n$, por

$$\frac{\partial}{\partial x_j} F = \sum_{i=1}^k \frac{\partial}{\partial x_i} g \frac{\partial}{\partial x_j} f_i \quad (4.6)$$

✓ Ejemplo 4.12

La composición de $\mathbf{f}(s, t) = (e^{s-t}, \cos(s^2t))$ y $g(x, y) = (x^2, x - y^3)$ da lugar a la función $F(s, t) = ((e^{s-t})^2, e^{s-t} - (\cos(s^2t)))$. Utiliza de nuevo la doble vía para comprobar la fórmula de las derivadas parciales en este caso.

4: Composición de Funciones. Regla de la Cadena is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

5: Aproximación Lineal de una Función Escalar de Varias Variables

This page is a draft and is under active development.

Recordemos que, bajo ciertas condiciones de derivabilidad, una función real puede aproximarse por una recta cuando se producen cambios pequeños (ε) en la variable independiente (**aproximación por la recta tangente**, figura 5),

$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x) \quad (5.1)$$

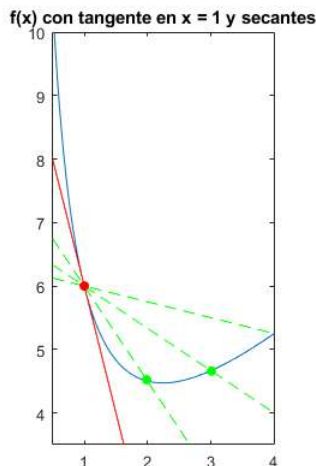


Fig. 5 - Aproximación de una función por su recta tangente

✓ Ejemplo 5.13

La función $f(x) = x^2$ puede aproximarse, en puntos cerca de $x = 1$ por la recta:

$$f(1 + \varepsilon) \approx f(1) + f'(1)\varepsilon = 1 + 2\varepsilon \quad (5.2)$$

Del mismo modo, para una función $f: \mathbb{R}^n \rightarrow \mathbb{R}$ que depende de varias variables, tendremos:

$$f(x_1 + \varepsilon_1, x_2 + \varepsilon_2, \dots, x_n + \varepsilon_n) \approx f(x_1, x_2, \dots, x_n) + \sum_{i=1}^n \varepsilon_i \frac{\partial f}{\partial x_i}(x_1, x_2, \dots, x_n) \quad (5.3)$$

Esta expresión puede escribirse de forma más compacta usando la definición de gradiente (Definición 4) y llamando $\varepsilon = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n)$, vector fila donde se agrupan los pequeños cambios que se producen en cada una de las variables independientes:

$$f(\mathbf{x} + \varepsilon) \approx f(\mathbf{x}) + \varepsilon \nabla f(\mathbf{x}) \quad (5.4)$$

En este caso, si tenemos $n = 2$, aproximamos la función cerca de un punto, por su plano tangente.

✓ Ejemplo 5.14

La función $f(x, y) = x^2 + xy + y^2$, en un entorno de $(1, 0)$, tomará valores cercanos a la aproximación de primer orden. Para calcularla necesitamos $f(1, 0)$ y $\nabla f(1, 0)$ que valen

- $f(1, 0) = 1$
- $\nabla f(1, 0) = (2x + y, x + 2y)|_{x=1, y=0} = (2, 1)$

y se emplean para determinar el plano tangente en el punto $(1, 0, f(1, 0))$, cuya ecuación es $2x + y = 2$. El vector normal al plano es $\nabla f(1, 0)$.

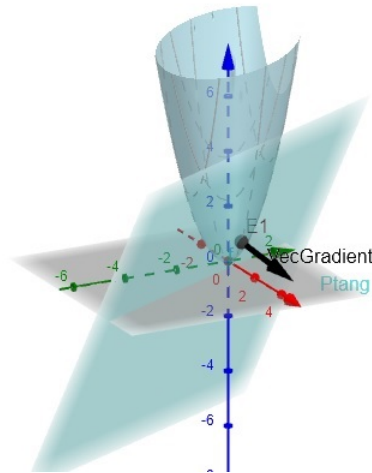


Fig. 6 - Aproximación de una función por su plano tangente

5: Aproximación Lineal de una Función Escalar de Varias Variables is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

6: Optimización y Método del Gradiente Descendiente

Vamos a describir el método del gradiente descendente, que sirve para encontrar un punto donde la función $f: \mathbb{R}^n \rightarrow \mathbb{R}$ alcanza un valor mínimo.

El proceso se puede resumir en:

- Comenzamos eligiendo un valor arbitrario inicial de $\mathbf{x}$.
- Buscamos la dirección $\mathbf{v}$ en la que f decrece más rápidamente alrededor de $\mathbf{x}$. A $\mathbf{v}$ le llamamos *dirección de máximo descenso*.
- Variamos “un poco” los valores iniciales en la dirección $\mathbf{v}$:

$$\mathbf{x} \rightarrow \mathbf{x} + \varepsilon \mathbf{v} \quad (6.1)$$

- Repetimos.

Nos detenemos ahora en el segundo paso de este proceso: cómo determinar la dirección de descenso más rápido. Como se ha visto en la sección anterior:

$$f(\mathbf{x} + \mathbf{v}) \approx f(\mathbf{x}) + \mathbf{v} \nabla f(\mathbf{x}). \quad (6.2)$$

Por la definición de producto escalar, suponiendo que $\mathbf{v}$ es un vector de módulo 1, la expresión anterior se escribe

$$f(\mathbf{x} + \mathbf{v}) \approx f(\mathbf{x}) + \mathbf{v} \nabla f(\mathbf{x}) = f(\mathbf{x}) + \|\nabla f(\mathbf{x})\| \cos(\theta), \quad (6.3)$$

donde θ es el ángulo que forman los vectores $\mathbf{v}$ y $\nabla f(\mathbf{x})$.

Con vistas a que f decrezca lo más rápido posible, debemos elegir θ de forma que $\cos(\theta)$ sea lo menor posible, esto es, $\cos(\theta) = -1$.

Esto ocurre cuando θ vale π (180°) o, lo que es lo mismo, si $\mathbf{v}$ **tiene el sentido opuesto a** $\nabla f(\mathbf{x})$.

El proceso queda reescrito como:

- Comenzamos eligiendo un valor arbitrario inicial de $\mathbf{x}$.
- La dirección $\mathbf{v}$ que hace descender más rápidamente a f alrededor de $\mathbf{x}$ es $-\nabla f(\mathbf{x})$. Calculamos $\nabla f(\mathbf{x})$.
- Variamos un poco los valores iniciales en la dirección $\mathbf{v}$:

$$\mathbf{x} \rightarrow \mathbf{x} - \varepsilon \nabla f(\mathbf{x}) \quad (6.4)$$

- Repetimos.

Para aplicar este algoritmo iterativo es necesario fijar dos parámetros: el número de iteraciones y el valor de ε (que se conoce como *tasa de aprendizaje*).

- Se dirá que el algoritmo converge si la sucesión de soluciones en el proceso iterativo se acerca a la solución óptima (esto es, el mínimo global). La cantidad de iteraciones que se necesita para que el método del gradiente descendente converja puede variar mucho (dependiendo del problema, desde unas pocas iteraciones hasta miles de ellas). En cualquier caso, la tasa de aprendizaje condicionará el número mínimo de iteraciones necesario para la convergencia.
- La tasa de aprendizaje ε es una constante, con $0 < \varepsilon < 1$, que afectará de forma significativa la efectividad del algoritmo: un valor demasiado pequeño puede llevar a que el algoritmo precise un número elevado de iteraciones antes de converger; mientras que si el valor asignado es demasiado grande, puede saltarse el mínimo global (pasando a tomar valores mayores) y no proporcionar una buena solución.

Sin embargo, este método no puede asegurar la convergencia al valor que minimiza globalmente la pérdida. La situación idílica sería la de una función de pérdida convexa, con un mínimo global bien definido (imaginémoslo como un paraboloide). Sin embargo, la mayoría de funciones de pérdida para modelos no-lineales son no convexas. Esto significa que pueden poseer varios mínimos locales, que no tienen por qué coincidir con el mínimo global. Al usar el método del gradiente descendente, si el algoritmo llega a un mínimo local, verá que aquí el gradiente es cero y que la pérdida crece en todas las direcciones; pero esto no significa que hayamos encontrado el mínimo global de la pérdida. Al empezar a iterar en un punto aleatorio, no tenemos garantía de que el

algoritmo vaya a ir descendiendo hacia el mínimo global. De hecho, lo más probable es que acabe convergiendo en un mínimo local (puesto que habrá más). Aquí es donde nace la necesidad de otros métodos que tomen la idea del gradiente descendente pero que lo lleven un poco más allá para solucionar este problema.

Método del gradiente descendente estocástico

Este método trata de solucionar el problema de la convergencia a un mínimo local añadiendo ruido al gradiente en cada iteración. De esta forma, en cada iteración, el gradiente se irá moviendo cuesta abajo en promedio; pero no en la dirección de máximo descenso. De hecho, podría incluso no moverse cuesta abajo en alguna iteración. Esto permite escapar de los mínimos locales para poder acabar llegando al mínimo global.

La forma de introducir aleatoriedad es sencilla: en cada iteración se usa solamente un subconjunto determinado de los datos de entrenamiento para actualizar los pesos; no utiliza todos. De esta forma la actualización quedará sesgada hacia ese subconjunto determinado, la pérdida con respecto a esos datos exclusivamente. Esta actualización puede no hacer que la pérdida **total** disminuya. La regla de actualización es:

$$\phi_{t+1} = \phi_t - \varepsilon \sum_{i \in B_t} \frac{\partial l_i[\phi]}{\partial \phi} \quad (6.5)$$

B_t es el subconjunto de datos que se usarán en cada iteración para actualizar la pérdida. En la práctica, es como si en cada iteración se estuviera minimizando una pérdida distinta. Si se elige ese subconjunto para que contenga todos los datos, recuperamos el método del gradiente descendente habitual. El lector podría preguntarse por qué este método, que debería ser peor puesto que usa menos datos, ofrece mejores resultados que el convencional. Existen una serie de propiedades interesantes:

1. Los elementos usados en cada iteración se eligen sin reemplazamiento, por lo que acaban contribuyendo todos los elementos por igual.
2. Es computacionalmente más eficiente usar este método puesto que solo se calcula el gradiente de un pequeño set de datos.
3. Permite escapar mínimos locales por el ruido añadido.
4. Reduce las posibilidades de atascarse en puntos de silla (donde el gradiente es cero, pero no son mínimos locales ni globales).

ADAM (Adaptive Moment Estimation)

Es otro método que trata de mejorar el método del gradiente descendente. En este caso lo hace añadiendo un **momento**, es decir, los parámetros se actualizan con una media ponderada de todos los gradientes anteriores. En particular, pondera el valor actual y el de la iteración previa (que también fue calculado usando los demás). La regla de actualización es la siguiente:

$$m_{t+1} = \beta m_t + (1 - \beta) \sum_{i \in B_t} \frac{\partial l_i[\phi]}{\partial \phi} \quad (6.6)$$

$$\phi_{t+1} = \phi_t - \varepsilon m_{t+1} \quad (6.7)$$

m_t es el momento en la t -ésima iteración y β controla cuánto se ponderan los gradientes anteriores con respecto al nuevo. Cada paso es una suma ponderada de los gradientes anteriores. Esto permite tener una trayectoria más suave y un menor comportamiento oscilatorio en los valles. Vemos que la suma sigue siendo sobre un subconjunto, así que funciona igual que el gradiente descendente estocástico, heredando sus propiedades.

Otro añadido que hace ADAM es solucionar el siguiente problema: gradientes mayores llevan a mayores ajustes de los parámetros (cuando quizás habría que ser más cauteloso) y gradientes pequeños llevan a ajustes muy pequeños en los parámetros (cuando quizás habría que explorar más allá). La solución a este problema es normalizar los gradientes para que, en cada paso, el avance esté dado siempre por la tasa de aprendizaje y el gradiente simplemente indique la dirección. Es decir:

$$v_{t+1} = \left(\frac{\partial L[\phi]}{\partial \phi} \right)^2 \rightarrow \phi_{t+1} = \phi_t - \varepsilon \frac{m_{t+1}}{\sqrt{v_{t+1} + \varphi}} \quad (6.8)$$

La suma de φ es para evitar la división por cero cuando el módulo del gradiente es muy pequeño. Ahora estamos realizando una media ponderada sobre la historia de todos los gradientes. Otra posible mejora es añadir momento (considerar los previos

valores) tanto en el gradiente como en su cuadrado:

$$m_{t+1} = \beta m_t + (1 - \beta) \sum_{i \in B_t} \frac{\partial l_i[\phi]}{\partial \phi} \quad (6.9)$$

$$v_{t+1} = \gamma v_t + (1 - \gamma) \sum_{i \in B_t} \left(\frac{\partial l_i[\phi]}{\partial \phi} \right)^2 \quad (6.10)$$

Un problema que se plantea ahora es que siempre estamos usando la misma ponderación de los nuevos datos: para las primeras iteraciones (donde tenemos muy pocos datos y debería ponderarse mucho el nuevo valor) y para las últimas (donde ya van muchos datos calculados y tendrían que ponderar poco los nuevos datos). Esta ponderación se incorpora mediante:

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta^{t+1}} \quad (6.11)$$

$$\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \gamma^{t+1}} \quad (6.12)$$

$$\phi_{t+1} = \phi_t - \varepsilon \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \varphi}} \quad (6.13)$$

Este algoritmo converge más fácilmente al mínimo global de la pérdida y se mueve adecuadamente en todas las direcciones del espacio de parámetros.

6: Optimización y Método del Gradiente Descendiente is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

7: Redes Neuronales

This page is a draft and is under active development.

[7.1: ¿Qué es una red neuronal?](#)

[7.2: Perceptrón](#)

[7.3: Funciones de Activación](#)

[7.4: Funciones de Pérdida](#)

[7.5: Perceptrón Multicapa](#)

[7.6: Ejemplo- Regresión lineal](#)

7: [Redes Neuronales](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

7.1: ¿Qué es una red neuronal?

Una vez conocidos los fundamentos y herramientas matemáticos que se encuentran tras la inteligencia artificial, es la hora de entender la necesidad de este formalismo para obtener una red neuronal.

Definición: Red neuronal

Una red neuronal es una función multivariable $f(\mathbf{x})$ que toma un vector como input un vector $\mathbf{x}$ y proporciona como salida otro vector $\mathbf{y}$. La forma de obtener los valores de salida es pasar los inputs a través de la red neuronal:

$$\mathbf{y} = f(\mathbf{x})$$

Nótese que esta descripción tan sencilla de una red neuronal es sumamente amplia. El vector $\mathbf{x}$ puede representar multitud de situaciones con distintas aplicaciones. La mejor forma de entenderlo es mediante algunos ejemplos.

Por ejemplo, el vector $\mathbf{x}$ podría ser un vector cuatridimensional incluyendo la superficie de una casa, el número de habitaciones, el número de baños y el número de plazas de garaje; y tras pasarlo por la red neuronal podría obtenerse un vector unidimensional $\mathbf{y}$ que represente el precio de dicha casa.

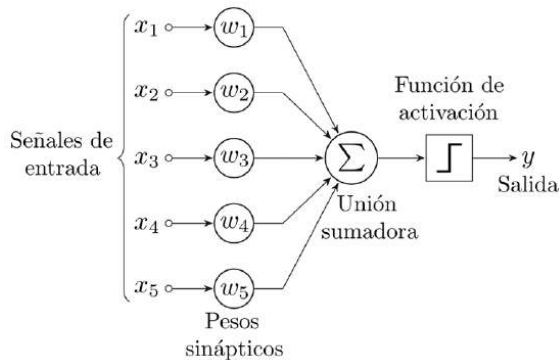
Supongamos ahora que queremos usar una red neuronal para determinar a qué género musical pertenece una canción. En este caso el vector de input $\mathbf{x}$ serían las distintas frecuencias a lo largo de la canción y su intensidad. La red neuronal habría sido entrenada para saber a qué rango de frecuencias pertenece cada género musical (el cómo entrenar la red lo veremos más adelante). Posteriormente el output de la red neuronal podría ser un vector $\mathbf{y}$ cuya dimensión sea el número de géneros a determinar (diez géneros entre rock, pop, electrónica...) y que sea un vector cuya única componente no nula sea el género al que corresponde la canción. Hemos resuelto un problema (determinar el género musical de una canción) que parecía tener poco que ver con el cálculo multivariable mediante un proceso que depende totalmente de él.

7.1: ¿Qué es una red neuronal? is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

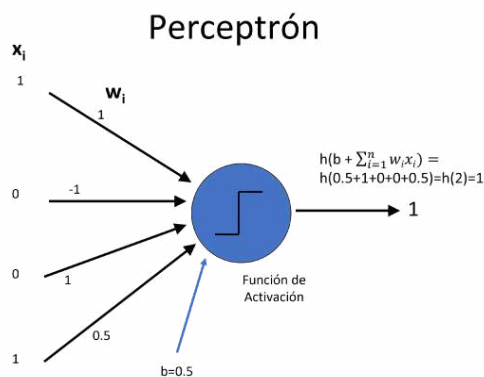
7.2: Perceptrón

El perceptrón fue propuesto en 1958 por [Frank Rosenblatt](#) en un proyecto del [laboratorio de aeronáutica de la Universidad de Cornell](#) financiado por la oficina de investigación naval de los EEUU. Se planteó inicialmente como una máquina, más que un programa o un algoritmo, dado que los pesos que se iban optimizando se actualizaban con motores eléctricos y potenciómetros.

En la actualidad, se considera el perceptrón como la unidad fundamental de una red neuronal, siendo el esquema principal del perceptrón el que se muestra a continuación:



Es decir, un perceptrón toma una serie de señales de entrada x , a las que multiplica por unos pesos w y obtiene la suma total de ese resultado. A ese valor se le aplica una función de activación y finalmente se obtiene un valor de salida. Ese valor de salida puede, a su vez, ser la entrada de otro perceptrón posterior. Un ejemplo de cálculo con cuatro valores de entrada, a los que se aplica una serie de pesos y un bias, y una función de activación de tipo escalón se muestra en la siguiente figura.



Existen muchas funciones de activación. Por ejemplo, una con valor de umbral b se puede definir como:

$$f\left(\sum_{j=1}^J w_j x_j\right) = \begin{cases} 1 & \text{si } \sum_{j=1}^J w_j x_j > b \\ 0 & \text{si } \sum_{j=1}^J w_j x_j \leq b \end{cases} = y$$

7.2: Perceptrón is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

7.3: Funciones de Activación

Las funciones de activación se aplican sobre el resultado final de cada perceptrón, permitiendo introducir efectos no lineales al cálculo. Ese es un aspecto fundamental a la hora de trabajar con redes neuronales. Sin estas funciones de activación, una red neuronal del tipo perceptrón multicapa se convertiría en una transformación lineal entre los datos de entrada y de salida, lo que limitaría mucho el tipo de casos que se podrían modelizar con redes neuronales.

La función escalón (binaria) y la función sigmoide fueron las primeras en proponerse y emplearse con éxito, pero se han ido proponiendo muchas otras a lo largo del tiempo. En la actualidad, la función RELU (que pone los número negativos de entrada a cero) es la más empleada.

Hay que tener en cuenta que en muchos casos interesa que sean funciones derivables, dado que se necesita para la optimización de la red. En puntos en lo que esto no sucede (Como RELU en el origen) se suelen usar aproximaciones numéricas para solventarlo.

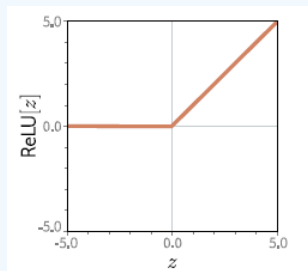
Merece la pena dedicar un tiempo a analizar las funciones de activación más relevantes en diversos problemas de aprendizaje automático.

✓ Ejemplo: ReLU (Rectified Linear Unit)

Es el tipo de función más sencilla que permite introducir no-linealidad en el modelo sin necesidad de transformaciones complicadas. Es la más usada y la más fácil de entender. Su definición es:

$$f(z) = \begin{cases} z & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$

Su representación gráfica es la siguiente:

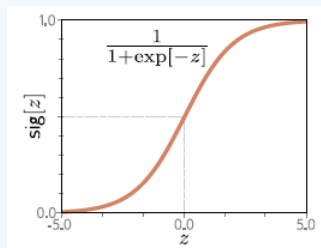


✓ Ejemplo: Sigmoide

Es útil para realizar clasificaciones binarias (clasificar el input entre solamente dos posibles categorías) y se define como:

$$f(x) = \frac{1}{1+e^{-x}}$$

Su representación gráfica es la siguiente:



Es fácil ver que la imagen de esta función es $y \in (0, 1)$. Esto permite realizar clasificaciones binarias como, por ejemplo: si el output es menor que 0.5 entonces el input pertenece a la primera clase; si el output es mayor que 0.5 pertenece a la segunda clase.

✓ Ejemplo: Softmax

La función softmax es útil para realizar clasificaciones multiclase, en las que se busca clasificar el output entre K categorías diferentes. Esta función toma como input un vector de longitud K (el output de la red neuronal, con sus pesos) y devuelve un vector de la misma longitud K pero ahora con todos los elementos en el intervalo [0,1]. Es una forma de transformar un vector de números reales en una distribución de probabilidad. Dado que los elementos de este vector (el output de softmax) se pueden entender como probabilidades, la suma de todos los elementos debe ser uno. Para asegurar la positividad de cada elemento y que la suma de los elementos sea 1, el elemento k-ésimo del output de la función softmax puede definirse como:

$$\text{softmax}_k[\mathbf{z}] = \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}$$

7.3: Funciones de Activación is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

7.4: Funciones de Pérdida

Consideramos que estamos trabajando con aprendizaje supervisado, en el que se dispone de datos de referencia de entrada y salida, y nuestro objetivo es entrenar una red para que aprenda a realizar el mapeo entre ambos conjuntos. Las funciones de pérdida, funciones de coste, o funciones de error, determinan cómo cuantificamos las diferencias entre las salidas de la red neuronal (estimaciones de los valores) que denominaremos (p), y las medidas de referencia (y). El entrenamiento de la red consistirá en la minimización de esta función, que denominaremos $L[\phi]$.

Existen muchas posibles formas de definir esta función de pérdida, pero es importante seleccionar la más apropiada para cada problema, en función del tipo de datos, el problema que estemos afrontando (clasificación, regresión...) así como el tipo de error estadístico que puedan tener nuestras medidas. Una adecuada selección de la función de pérdida es fundamental para obtener una correcta solución.

Un listado completo de las funciones de pérdida se puede consultar en referencias como "[Loss Functions](#)". Algunas de las funciones de pérdida más comunes son:

1) Mean Squared Error: Es la forma más tradicional de calcular la desviación. Se calcula como:

$$MSE = \sum_i |\mathbf{y}_i - \mathbf{p}_i|^2$$

2) Mean Average Error: Es parecida al MSE, pero tiene la ventaja de que las desviaciones más grandes entre el valor que arroja la red neuronal y el valor esperado contribuyen menos, al no estar elevado al cuadrado.

$$MAE = \sum_i |\mathbf{y}_i - \mathbf{p}_i|$$

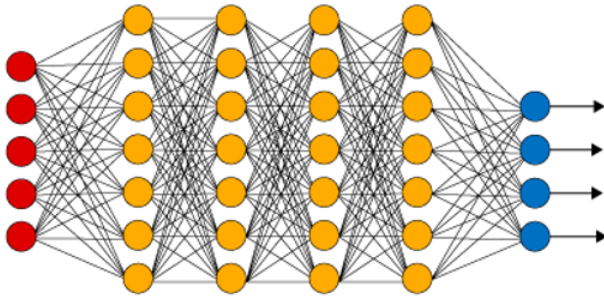
3) Binary Cross Entropy (BCE): Entropía binaria cruzada, especialmente útil en redes que se usan para clasificación binaria (en casos de clasificación multiclase, se denomina entropía cruzada multiclase). Recordemos que para este tipo de problemas se usaba la función sigmoide como función de activación, generalmente.

$$BCE = \sum_i -(1 - y_i) \log[1 - \text{sig}(f(x_i, \phi))] - y_i \log[\text{sig}(f(x_i, \phi))]$$

7.4: Funciones de Pérdida is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

7.5: Perceptrón Multicapa

El perceptrón multicapa consiste en la concatenación de capas de perceptrones en un esquema similar al que se muestra a continuación:



En este esquema de una red neuronal del tipo perceptrón multicapa, las entradas figuran en rojo, las salidas en azul, y los distintos perceptrones en amarillo.

Como se puede observar, en este tipo de red neuronal (denominado feed-forward en inglés), los perceptrones se organizan en capas (las distintas columnas en la figura), de manera que los perceptrones o neuronas de cada capa realizan operaciones de manera simultánea sobre los datos precedentes (ya sean de entrada o de una capa previa). El alto número de capas típico de estas redes que se emplea en problemas complejos es lo que llevó a calificarlas como "redes profundas" y derivó en el concepto de "Aprendizaje profundo" o Deep Learning.

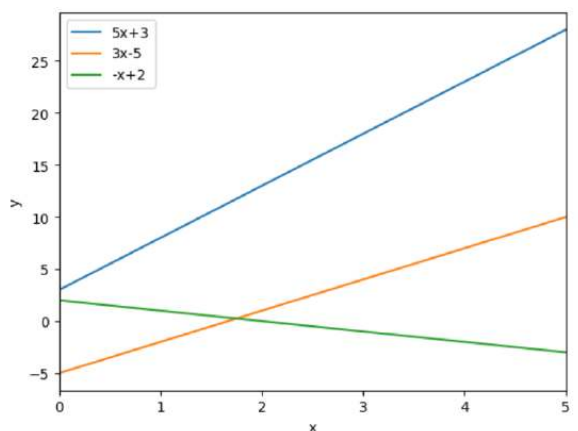
Existen muchos otros tipos de redes neuronales, que se diferencian principalmente en la forma en la que se establecen las conexiones entre los perceptrones o neuronas de la red, estableciendo conexiones recursivas, saltando una serie de capas.. El diseño de las conexiones óptimas de los perceptrones de una red es un importante campo de investigación en la actualidad.

7.5: Perceptrón Multicapa is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

7.6: Ejemplo- Regresión lineal

La mejor manera de entender el algoritmo de entrenamiento de una red neuronal es mediante un ejemplo sencillo. Empezamos con una tarea sencilla a la par que común dentro del mundo de las ciencias: la regresión lineal. Aproximar un conjunto de datos por una recta es un proceso que todo científico acabará realizando en algún momento, y este proceso puede entenderse como el entrenamiento de una red neuronal.

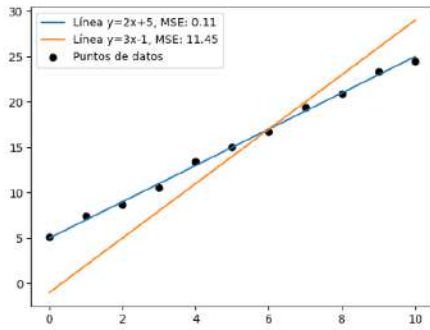
En este caso, nuestra red o modelo toma un input x y devuelve un output y que se calcula de forma lineal: $y = f(x, \phi) = \phi_0 + \phi_1 x$. El conjunto de parámetros ϕ determinan una familia de rectas en el plano con distintas pendientes y distintas ordenadas en el origen. Sin embargo, existe una elección óptima de dichos parámetros que es la óptima para ajustar un determinado conjunto de datos x, y . Se incluye a continuación una serie de rectas en el intervalo $[0, 5]$, que solo difieren en los valores de ϕ .



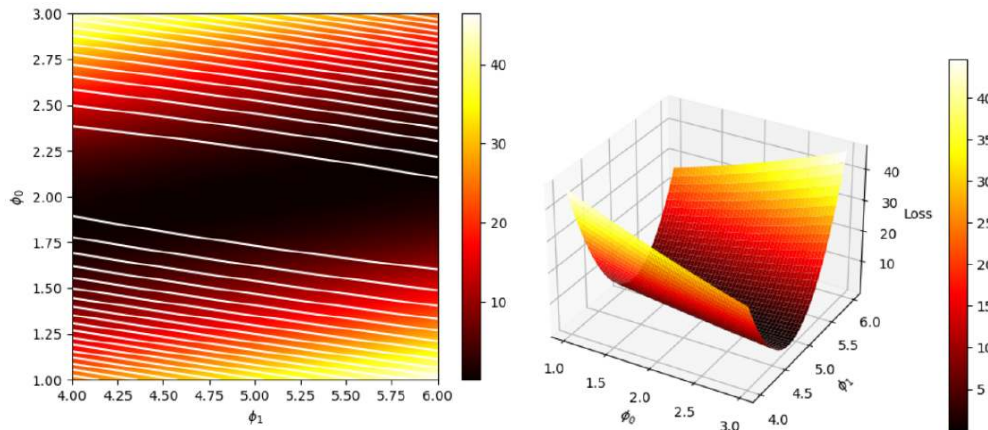
Vayamos al ejemplo concreto. Imaginemos que tenemos un set de datos, queremos encontrar la recta que mejor se ajusta a nuestros datos. Aquí entra el primer problema: ¿qué es ajustarse bien? En este caso puede parecer sencillo, pero en situaciones más complejas, ajustarse bien puede ser una definición complicada. Esta pregunta es equivalente a una pregunta ya tratada en otra sección: ¿qué función de pérdida debo elegir? En este caso, dado que estamos tratando con puntos en el plano y queremos minimizar su distancia a una recta; podemos definir como función de pérdida dicha distancia geométrica entre puntos. Por tanto, el error cuadrático medio o el error absoluto medio podrían ser buenos candidatos, pues ambos cuantifican la distancia geométrica entre los puntos de nuestra recta y los de nuestro set. Convencionalmente se usa el **error cuadrático medio**, ajuste que se denomina **ajuste de mínimos cuadrados**. Este nombre viene dado por la elección de función de pérdida, que en este caso tendría la forma:

$$L[\phi] = \sum_{i=1}^N (f(x_i, \phi) - y_i)^2 = \sum_{i=1}^N (\phi_0 + \phi_1 x_i - y_i)^2 \quad (7.6.1)$$

Podría haberse elegido como función de pérdida simplemente la suma del valor absoluto de la desviación (sin elevarse al cuadrado), pero convencionalmente se eleva al cuadrado para que las desviaciones mayores sumen aún más; de esta forma haciendo que mayores desviaciones hagan crecer mucho la pérdida. Veamos un ejemplo. Puede verse claramente que la recta que mejor se ajusta es aquella con menor pérdida.



Una vez fijados los puntos, la pérdida es una función de únicamente ϕ_o y ϕ_1 . Veamos una representación de esta función bidimensional, con sus correspondientes curvas de nivel, usando unos valores de $\phi_o \in [1, 3]$ y $\phi_1 \in [4, 6]$.



Para obtener este mapa simplemente hay que evaluar la pérdida para todas las posibles combinaciones de ϕ_o y ϕ_1 . Podemos ver que los valores menores de la pérdida se encuentran alrededor de $\phi_o \approx 2$ y $\phi_1 \approx 5$, como predecíamos. El valor óptimo de los parámetros de la red se encuentra en la región coloreada en negro. Nuestro objetivo será acercarnos al máximo global de la función todo lo posible. Para ello usaremos el método del gradiente descendiente estocástico (esto último para asegurar la convergencia). Lo primero que necesitaremos será el gradiente de la pérdida, para poder posteriormente trabajar con él y minimizar la pérdida. En nuestro caso el gradiente puede ser calculado de forma analítica:

$$\vec{\nabla} L = \frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^N l_i = \sum_{i=1}^N \frac{\partial l_i}{\partial \phi} \quad (7.6.2)$$

L es la pérdida total, l_i es la pérdida correspondiente a cada punto del set usado para entrenar. El gradiente de la pérdida para cada punto puede calcularse también:

$$\frac{\partial l_i}{\partial \phi} = \begin{pmatrix} \frac{\partial l_i}{\partial \phi_o} \\ \frac{\partial l_i}{\partial \phi_1} \end{pmatrix} = \begin{pmatrix} 2(\phi_o + \phi_1 x_i - y_i) \\ 2x_i(\phi_o + \phi_1 x_i - y_i) \end{pmatrix} \quad (7.6.3)$$

Ya solamente queda implementar el método del gradiente descendiente (ignoraremos la parte estocástica de momento). La regla de actualización de los parámetros en cada iteración será:

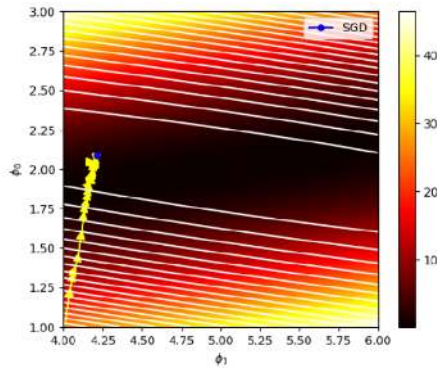
$$\phi_t = \phi_{t-1} - \epsilon \frac{\partial L}{\partial \phi} \quad (7.6.4)$$

Por componentes, esta ecuación puede escribirse como:

$$\phi_{o,t} = \phi_{o,t-1} - 2\epsilon \sum_{i=1}^N (\phi_{o,t-1} + \phi_{1,t-1} x_i - y_i) \quad (7.6.5)$$

$$\phi_{1,t} = \phi_{1,t-1} - 2\epsilon \sum_{i=1}^N x_i (\phi_{0,t-1} + \phi_{1,t-1} x_i - y_i) \quad (7.6.6)$$

Si se eligen unos valores iniciales de ambos parámetros, puede procederse al cálculo iterativo del mínimo de la pérdida. Debe elegirse de forma sensata la tasa de aprendizaje ϵ para que las iteraciones no se desvíen demasiado del mínimo. Sin embargo, para asegurar la convergencia, se usará tan solo un pequeño subconjunto de los datos disponibles en cada iteración, elegidos de forma aleatoria (gradiente descendiente estocástico). Usando tan solo 30 iteraciones y una tasa de aprendizaje de 0.002, pudo obtenerse el siguiente ajuste:



Podemos ver que, a medida que el método va progresando, se va acercando a la zona de menor pérdida. Los valores finales que se han obtenido son $\phi_0 \approx 2.01$ y $\phi_1 \approx 4.21$; que entran dentro del rango de lo esperado (aunque podría llegarse a un valor más preciso usando otro optimizador, tasa de aprendizaje o número de iteraciones). El modelo ha sido entrenado satisfactoriamente y ahora podría usarse para predecir el output esperado y dado un input x usando los parámetros calculados.

7.6: Ejemplo- Regresión lineal is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

8: Tipos de redes neuronales

8.1: Redes neuronales superficiales

8.2: Redes neuronales profundas

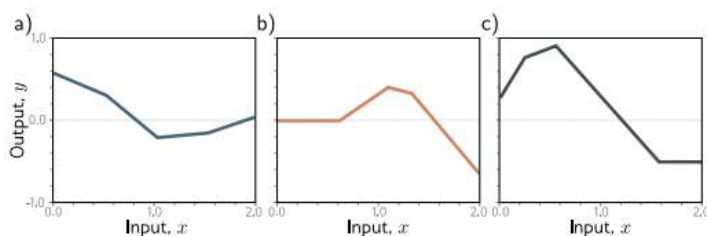
8: Tipos de redes neuronales is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

8.1: Redes neuronales superficiales

Las redes neuronales superficiales son aquellas redes neuronales con una sola **capa oculta**. Una red neuronal superficial tan solo tiene 3 capas: la capa de entrada, la capa oculta (donde se encuentran los pesos y se realiza el entrenamiento) y una capa de salida. Empezaremos viendo un ejemplo para un caso particular y posteriormente generalizaremos el resultado. Imaginemos una red neuronal con 10 parámetros, de forma que tome un input x y devuelva un output:

$$y = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x) + \phi_3 a(\theta_{30} + \theta_{31}x) \quad (8.1.1)$$

$a(x)$ es la función de activación que elijamos, nosotros para este ejemplo usaremos ReLU. La combinación de diversos parámetros en la ecuación anterior determina una familia de funciones, veamos un ejemplo:



Fuente: Understanding Deep Learning, Simon JD Prince

El cambio en los parámetros puede cambiar la altura de cada región, la posición de los vértices o las pendientes de cada recta.

El funcionamiento de la red neuronal puede reescribirse y reestructurarse en función de tres **unidades ocultas** que son independientes entre sí. Definimos:

$$h_1 = a(\theta_{10} + \theta_{11}x) \quad (8.1.2)$$

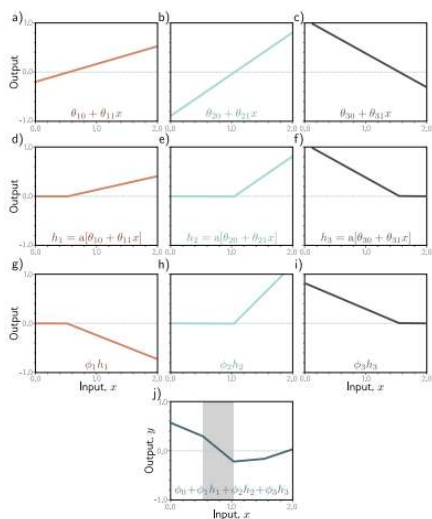
$$h_2 = a(\theta_{20} + \theta_{21}x) \quad (8.1.3)$$

$$h_3 = a(\theta_{30} + \theta_{31}x) \quad (8.1.4)$$

Cada h_i se denomina unidad oculta. La combinación lineal de las 3 unidades ocultas (más un bias) nos da el efecto de la red neuronal:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3 \quad (8.1.5)$$

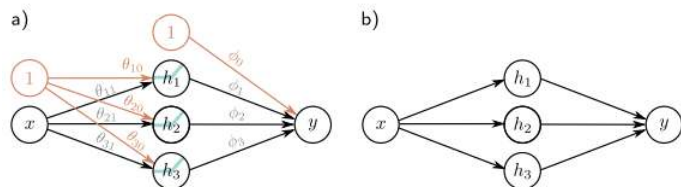
Cada unidad oculta toma una función lineal y posteriormente la pasa por una función de activación; luego los outputs de cada unidad oculta son sumados con un cierto peso y se añade un bias ϕ_0 que controla la altura de la función. Veamos el ejemplo con el desglose de cada unidad oculta, y posteriormente su suma:



La primera fila muestra las funciones lineales (vemos tres rectas normales); la segunda fila muestra las rectas pasadas por el ReLU (solo nos quedamos con la parte positiva) y la tercera fila muestra las rectas pasadas por el ReLU ponderadas con un cierto ϕ_i que modifica la pendiente del tramo que no es cero. Por último, se muestra la suma de la tercera fila. Podemos ver que los 3 puntos donde cada unidad oculta se hace cero representan los vértices de la función final. Al haber 3 unidades ocultas, la función tendrá 3 vértices y por tanto podrá dividirse en $3+1=4$ zonas. En general, si hay N unidades ocultas, la gráfica final tendrá N vértices y podrá dividirse en $N+1$ zonas. Fijémonos ahora en la zona gris de la última figura. Podemos ver que solamente están contribuyendo al modelo final las unidades h_1 y h_3 , decimos que estas unidades están **activas**, mientras que la unidad h_2 no está contribuyendo porque el ReLU la ha hecho cero, decimos que está **inactiva**. La pendiente de la recta en esta zona gris es la combinación lineal de las

pendientes de las unidades activas, pesadas con los ϕ_i : $\phi_1 \theta_{11} + \phi_3 \theta_{31}$. Cada zona se denomina **región lineal**.

La arquitectura de esta red puede resumirse en la siguiente figura:



suele ser muy alto y esta representación resulta más sencilla e intuitiva.

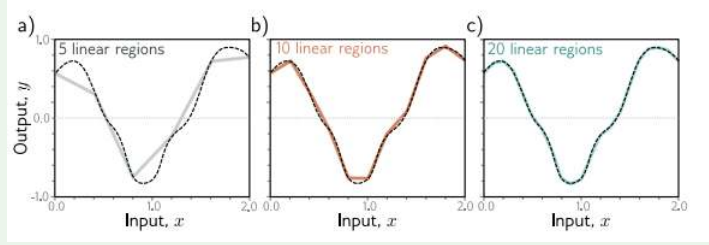
La figura de la izquierda muestra de forma explícita todos los parámetros que se están utilizando (mediante cajas naranjas que representa la incorporación de estos parámetros como otra unidad), así como la jerarquía de las unidades ocultas y su posterior combinación. En la práctica resulta más conveniente usar esquemas como el de la segunda figura, puesto que el número de parámetros

Una vez entendido este sencillo ejemplo de red con 3 unidades ocultas, es necesario introducir un teorema que justifique el uso de estas funciones ReLU y que solamente usemos rectas.

Teorema: Teorema de Aproximación Universal

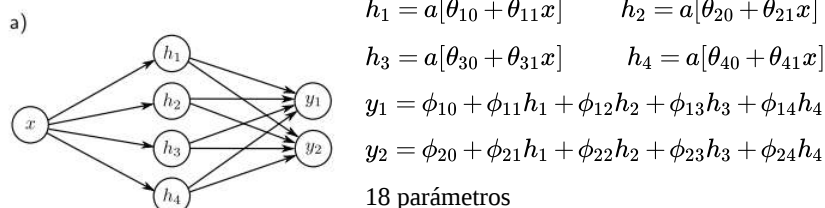
Toda función continua que mapea intervalos de números reales a algún intervalo de salida de números reales puede ser aproximado por una red neuronal superficial (una sola capa oculta) a cualquier precisión deseada.

Este teorema puede resultar chocante pero es en realidad un resultado bastante intuitivo, y la mejor forma de verlo es mediante un ejemplo. Se muestra a continuación una función arbitraria y la aproximación mediante una red neuronal superficial (funciones lineales definidas a trozos) con 5, 10 y 15 regiones lineales (es decir, usando 4, 9 y 14 unidades ocultas):

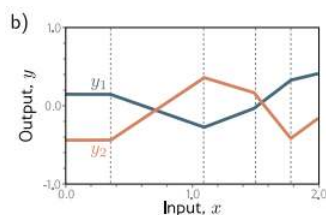


Vemos que la aproximación a la función arbitraria es mejor cuanto mayor es el número de regiones lineales. De esta forma, podría alcanzarse cualquier precisión en la aproximación simplemente añadiendo más unidades ocultas.

Veamos ahora que una red neuronal superficial también puede tener más de un output, y los efectos que tiene esto. Se incluye a continuación el funcionamiento y arquitectura de la red:

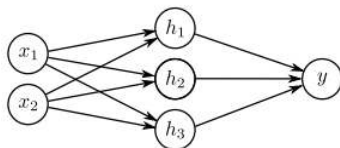


Vemos ahora que, al tener 4 unidades ocultas, tenemos 4 vértices y 5 regiones lineales. Cada output tiene su propio comportamiento y pendiente, pero ambas comparten los mismos vértices porque ambas son la combinación lineal de las mismas unidades ocultas: el ReLU ha generado un vértice en el mismo punto para ambas.



Hemos podido comprobar que la dimensión del output NO modifica el número de regiones disponibles, que sigue siendo función solo del número de unidades ocultas.

Comprobaremos ahora que este NO es el caso cuando modificamos las dimensiones del input. Supongamos ahora una red con la siguiente arquitectura:



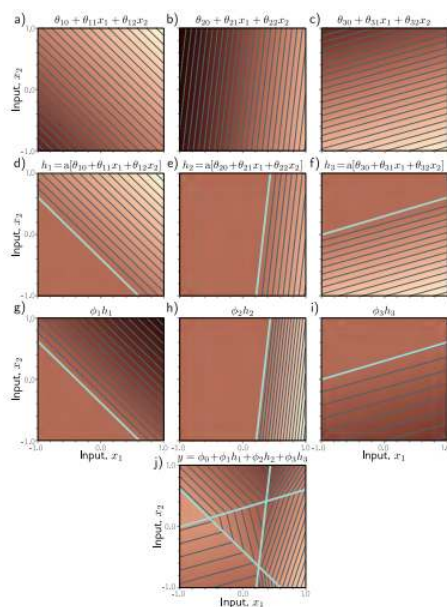
$$h_i = a[\theta_{i0} + \theta_{i1}x_1 + \theta_{i2}x_2]$$

El input de la función de activación ya no es una recta, sino un **plano**. Al pasarlo por la función de activación, se mapean a cero aquellos valores de z negativos, definiendo z como $z = \theta_{i0} + \theta_{i1}x_1 + \theta_{i2}x_2$.

ReLU ya no genera vértices entre regiones lineales sino

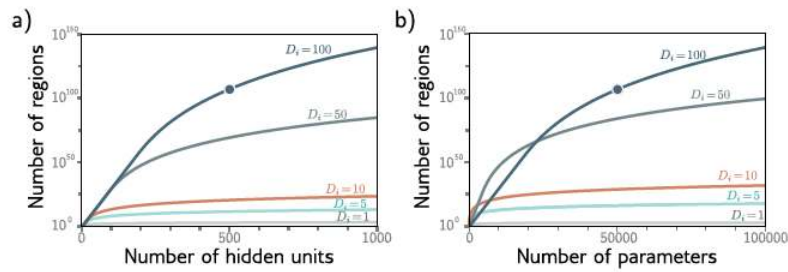
entre **regiones poligonales**.

Como siempre, tratemos de visualizar esto mediante un ejemplo. Se muestra a continuación el desglose de esta red neuronal superficial:



La primera fila representa los planos correspondientes a cada unidad oculta. La segunda fila representa los planos mapeados, con las zonas negativas mapeadas a 0 por el ReLU. La tercera fila representa el plano mapeado pesado con su correspondiente ϕ_i . Por último, la última imagen representa la combinación lineal de la tercera, es una superficie lineal continua que consiste de regiones poligonales convexas. Cada región tiene unas unidades activas u otras. En este caso hemos pasado de regiones 2D (1 dimensión input) a regiones 3D (2 dimensiones input 1 dimensión output).

Obviamente, si seguimos aumentando el número de dimensiones output, pasaremos a representaciones en más dimensiones que ya no podemos visualizar. Para el caso de tener el mismo número de unidades ocultas D y dimensiones del input D_i , podríamos alinear cada hiperplano con uno de los ejes. Para dos dimensiones de input podemos dividir el espacio en cuatro cuadrantes; para tres dimensiones de input podemos dividir el espacio en ocho octantes... para D_i dimensiones se crean 2^{D_i} ortantes. Es común que las redes neuronales tengan más unidades ocultas que dimensiones del input, así que crean más de 2^{D_i} regiones lineales. Se incluye a continuación una representación del número de regiones del espacio input-output para distinto número de unidades ocultas y de dimensiones del input. Podemos ver que crece muy rápidamente, teniendo $D=500$ unidades ocultas y $D_i = 100$ dimensiones del input, podemos tener 10^{107} regiones lineales.



Pasemos ahora al **caso general** para una red neuronal superficial que toma inputs $x \in \mathbb{R}^{D_i}$ y da outputs $y \in \mathbb{R}^{D_o}$ usando $h \in \mathbb{R}^D$. Cada unidad oculta funciona como:

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right] \quad (8.1.6)$$

Estas unidades se combinan linealmente para crear el output:

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d \quad (8.1.7)$$

8.1: Redes neuronales superficiales is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

8.2: Redes neuronales profundas

8.2: Redes neuronales profundas is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

9: Retropropagación

This page is a draft and is under active development.

Idea intuitiva de la retropropagación

El algoritmo de retropropagación es un método muy utilizado en redes neuronales para minimizar la conocida como función de pérdida o de error de la red. Esta función proporciona, durante el entrenamiento de la red neuronal, una medida de cuánto se desvían los valores proporcionados por la red de los valores reales o deseados. Los parámetros desconocidos de la red, que son las incógnitas del problema y suelen denominarse pesos, se modifican de tal forma que los valores de salida de la red se ajusten lo más posible a los deseados. Para ello, se utiliza el método del gradiente descendente, que permite minimizar el error en función de los pesos. Puesto que el error se calcula al final de la red (en la capa de salida) es más fácil calcular el efecto de las variaciones de los pesos (y, por tanto, ajustarlos) comenzando desde el final, esto es, cambiando los pesos que llegan a la capa de salida. Sucesivamente, se modifican los de cada capa intermedia en dirección hacia la capa inicial. En este proceso será de utilidad la regla de la cadena.

✓ Ejemplo 9.15

Vamos a considerar las funciones escalares $f(x) = x^2$, $g(y) = \sin y$ y $h(z) = \exp(-z)$. La composición de las tres funciones $(h \circ g \circ f)(x)$ da lugar a $J(x) = \exp(-\sin x^2)$. Vamos a calcular $\frac{dJ}{dx}$ utilizando el algoritmo de retropropagación. Aunque la composición de funciones se realiza hacia delante:

$$x \rightarrow y = f(x) = x^2 \rightarrow z = g(y) = \sin y \rightarrow J(x) = h(z) = \exp(-z) \quad (9.1)$$

el cálculo de la derivada lo realizamos hacia atrás, utilizando la regla de la cadena:

1. Empezamos por $h(z) = \exp(-z)$:

$$\frac{dJ}{dz} \rightarrow -\exp(-z) \quad (9.2)$$

2. Después por $z = g(y) = \sin(y)$:

$$\frac{dJ}{dy} \rightarrow \frac{dJ}{dz} \frac{dz}{dy}, \quad \frac{dz}{dy} = \cos(y) \quad (9.3)$$

3. Por último por $y = f(x) = x^2$:

$$\frac{dJ}{dx} \rightarrow \frac{dJ}{dy} \frac{dy}{dx}, \quad \frac{dy}{dx} = 2x \quad (9.4)$$

4. Así, resulta

$$\frac{dJ}{dx}(x) = -2x \cos(x^2) \exp(-\sin(x^2)) \quad (9.5)$$

✓ Ejemplo 9.16

Sean $\mathbf{x} = (x_1, x_2)$ y las funciones: $f(x_1, x_2) = a_1 x_1 + a_2 x_2$ con a_1 y a_2 constantes, $g(y) = 1 + e^{-y}$ y $h(z) = \frac{1}{z}$. La composición de las tres funciones $(h \circ g \circ f)(x)$ da lugar a $J(x_1, x_2) = \frac{1}{1 + e^{-(a_1 x_1 + a_2 x_2)}}$. Esto es:

$$(x_1, x_2) \rightarrow y = f(x_1, x_2) = a_1 x_1 + a_2 x_2 \rightarrow z = g(y) = 1 + e^{-y} \rightarrow J(\mathbf{x}) = h(z) = \frac{1}{z} \quad (9.6)$$

El objetivo en este caso es el cálculo del gradiente de la función J , $\nabla J(\mathbf{x})$, que tiene por coordenadas $\frac{\partial}{\partial x_i} J(\mathbf{x})$, para $i = 1, 2$.

1. Empezamos por $h(z) = \frac{1}{z}$:

$$\frac{\partial J}{\partial z} \rightarrow -\frac{1}{z^2} \quad (9.7)$$

2. Después tomamos $z = g(y) = 1 + e^{-y}$:

$$\frac{\partial J}{\partial y} \rightarrow \frac{\partial J}{\partial z} \frac{\partial z}{\partial y}, \quad \frac{\partial z}{\partial y} = -e^{-y} \quad (9.8)$$

3. Por último, para $y = f(x_1, x_2) = a_1 x_1 + a_2 x_2$:

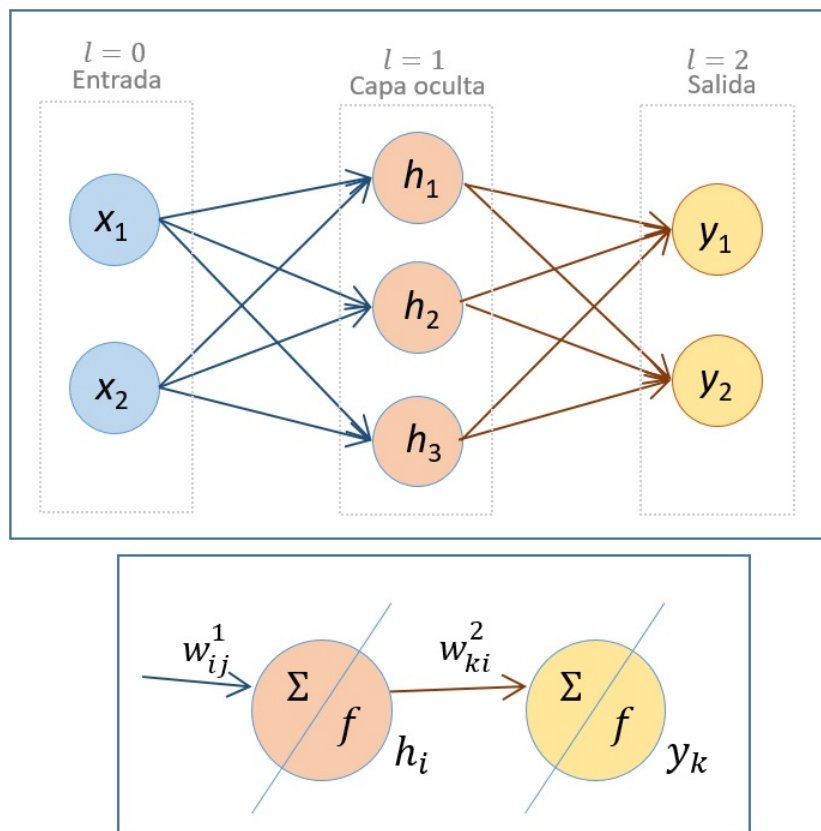
- $\frac{\partial J}{\partial x_1} \rightarrow \frac{\partial J}{\partial y} \frac{\partial y}{\partial x_1}, \quad \frac{\partial y}{\partial x_1} = a_1$
- $\frac{\partial J}{\partial x_2} \rightarrow \frac{\partial J}{\partial y} \frac{\partial y}{\partial x_2}, \quad \frac{\partial y}{\partial x_2} = a_2$

9: Retropropagación is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

10: Entrenamiento de una Red Neuronal

This page is a draft and is under active development.

En lo que sigue, vamos a analizar el proceso de entrenamiento en una RNA de *dos capas* (ver figura 3). La red consta de una capa de entrada (correspondiente a $l = 0$, con dos neuronas), una capa intermedia *oculta* (correspondiente a $l = 1$, con tres neuronas) y la capa de salida ($l = 2$ y dos neuronas). El objetivo es calcular los pesos de la RNA, utilizando el algoritmo de retropropagación de la siguiente forma:



Ejemplo de red neuronal

- Las entradas (en $l = 0$), $\mathbf{x} = (x_1, x_2)^T$ son valores constantes. Vamos a denotar por h_i la salida de la neurona i de la capa $l = 1$, que serán a su vez entradas de la capa $l = 2$. Análogamente, denotamos por y_1 e y_2 las salidas de la capa $l = 2$ (salidas de la RNA).
- Los parámetros que debemos ajustar son los pesos (enlaces entre neuronas de diferentes capas). Se denotan por w_{ij}^l , donde el superíndice l hace referencia a la capa a la que llega y los subíndices i y j a las neuronas de las capas l y $l - 1$, respectivamente, que conecta este peso.
- En las neuronas de la capa oculta y de la capa de salida se realizan dos procesos:
 - El primero, que simbolizamos con Σ , corresponde en ambas capas a la combinación lineal de la salida de la capa anterior con los pesos correspondientes. Esto es:
 - En la neurona i ($i = 1, 2, 3$) de la capa $l = 1$: $a_i^1 = x_1 w_{i1}^1 + x_2 w_{i2}^1$.
 - En la neurona k ($k = 1, 2$) de la capa $l = 2$: $a_k^2 = h_1 w_{k1}^2 + h_2 w_{k2}^2 + h_3 w_{k3}^2$.
 - El segundo, que representamos en la figura con f , corresponde a la función de activación que suponemos que es la misma para $l = 1, 2$:

$$\sigma(u) = \frac{1}{1 + e^{-u}} \quad (10.1)$$

Recordemos que la derivada de esta función es:

$$\sigma'(u) = \frac{e^{-u}}{(1 + e^{-u})^2} = \sigma(u)(1 - \sigma(u)) \quad (10.2)$$

Entonces, tenemos que las salidas de cada capa son:

- $h_i = \sigma(a_i^1), i = 1, 2, 3$
- $y_k = \sigma(a_k^2), k = 1, 2$

- Definimos la función de pérdida o de error, como se hace usualmente, mediante la desviación cuadrática media

$$L(y_1, y_2) = \frac{1}{2} \sum_{k=1}^2 (y_k - t_k)^2 \quad (10.3)$$

siendo t_k ($k = 1, 2$) los valores deseados correspondientes a cada salida y_k de la RNA.

Puesto que nuestro objetivo es minimizar la función de pérdida, debemos buscar los pesos que lo consiguen, utilizando el algoritmo de retropropagación. Se trata de un proceso iterativo en el que se parte de unos valores iniciales para los pesos y estos se van modificando mediante el método del gradiente descendente. Con los valores iniciales de los pesos, a partir de los valores $\mathbf{x}$ se calculan los de $\mathbf{a}^1, \mathbf{h}, \mathbf{a}^2$ e $\mathbf{y}$. Con ellos, partiendo de la capa de salida, vamos hacia atrás:

1. Aplicamos la regla de la cadena

$$\frac{\partial L}{\partial w_{ki}^2} \rightarrow \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial a_k^2} \frac{\partial a_k^2}{\partial w_{ki}^2} \quad (10.4)$$

con ($k = 1, 2$):

- $\frac{\partial}{\partial w_{ki}^2} a_k^2(w_{k1}^2, w_{k2}^2, w_{k3}^2) = h_i \quad (i = 1, 2, 3)$
- $\frac{\partial y_k}{\partial a_k^2} = \frac{\partial}{\partial a_k^2} \sigma(a_k^2) = \sigma(a_k^2)(1 - \sigma(a_k^2)) = y_k(1 - y_k)$
- $\frac{\partial}{\partial y_k} L(y_1, y_2) = (y_k - t_k)$

Resulta, sustituyendo:

$$\frac{\partial L}{\partial w_{ki}^2} \rightarrow (y_k - t_k) y_k(1 - y_k) h_i \quad (10.5)$$

2. Siguiendo el método del gradiente descendente, actualizamos los pesos w_{ki}^2 en esta capa. Llamando $\mathbf{w}^2 = (w_{ki}^2)_{k=1,2;i=1,2,3}$

$$\mathbf{w}^2 \leftarrow \mathbf{w}^2 - \varepsilon \nabla_{\mathbf{w}^2} L \quad (10.6)$$

3. Aplicamos la regla de la cadena en la capa oculta:

$$\frac{\partial L}{\partial w_{ij}^1} \rightarrow \frac{\partial L}{\partial h_i} \frac{\partial h_i}{\partial a_i^1} \frac{\partial a_i^1}{\partial w_{ij}^1} \quad (10.7)$$

con ($i = 1, 2, 3$):

- $\frac{\partial}{\partial w_{ij}^1} a_i^1(w_{i1}^1, w_{i2}^1) = x_j \quad (j = 1, 2)$
- $\frac{\partial h_i}{\partial a_i^1} = \frac{\partial}{\partial a_i^1} \sigma(a_i^1) = \sigma(a_i^1)(1 - \sigma(a_i^1)) = h_i(1 - h_i)$
- Para calcular $\frac{\partial L}{\partial h_i}$ tenemos en cuenta la dependencia de las funciones respecto a las distintas variables: $L(y_1, y_2), y_k(a_k^2)$ y $a_k^2(h_1, h_2, h_3)$. Así, aplicando la regla de la cadena:

$$\frac{\partial L}{\partial h_i} \rightarrow \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial a_1^2} \frac{\partial a_1^2}{\partial h_i} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial a_2^2} \frac{\partial a_2^2}{\partial h_i} \quad (10.8)$$

Ya habíamos obtenido anteriormente las expresiones para $\frac{\partial L}{\partial y_k}$ y $\frac{\partial y_k}{\partial a_k^2}$, por lo que sólo queda calcular el último factor de cada sumando:

$$\blacksquare \frac{\partial a_2^k}{\partial h_i} = w_{ki}^2$$

Sustituyendo:

$$\frac{\partial L}{\partial w_{ij}^1} \rightarrow x_j h_i (1 - h_i) \sum_{k=1}^2 [(y_k - t_k) y_k (1 - y_k) w_{ki}^2] \quad (10.9)$$

4. Finalmente, se actualizan los pesos w_{ij}^1 . Llamando $\mathbf{w}^1 = (w_{ij}^1)_{i=1,2,3:j=1,2}$

$$\mathbf{w}^1 \leftarrow \mathbf{w}^1 - \varepsilon \nabla_{\mathbf{w}^1} L \quad (10.10)$$

Con esto se ha dado un paso de la iteración de la actualización de los pesos. A continuación, se volvería a repetir el proceso partiendo de los valores de los datos $\mathbf{x}$.

En la práctica se suele contar para el entrenamiento con un conjunto de datos (no solo uno) y la función que se minimiza entonces es la suma de todas las “pérdidas” que se tienen al considerar todos esos datos.

10: Entrenamiento de una Red Neuronal is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

11: Apéndices

[11.1: Índice alfabético](#)

[11.2: Glosario](#)

[11.3: Detailed Licensing](#)

[11.4: Bibliografía Recomendada](#)

[11: Apéndices](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.1: Índice alfabético

C

composicion de funciones

[4: Composición de Funciones. Regla de la Cadena](#)

L

Loss Functions

[7.4: Funciones de Pérdida](#)

R

regla de la cadena

[4: Composición de Funciones. Regla de la Cadena](#)

11.1: Índice alfabético is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.2: Glosario

Sample Word 1 | Sample Definition 1

11.2: Glosario is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.3: Detailed Licensing

Overview

Title: [Las matemáticas de la inteligencia artificial](#)

Webpages: 32

All licenses found:

- [Undeclared](#): 100% (32 pages)

By Page

- [Las matemáticas de la inteligencia artificial](#) - *Undeclared*
 - [1: Portada](#) - *Undeclared*
 - [1.1: Titulo](#) - *Undeclared*
 - [1.2: Página de Información](#) - *Undeclared*
 - [1.3: Table of Contents](#) - *Undeclared*
 - [1.4: Licensing](#) - *Undeclared*
 - [2: Introducción a la Inteligencia Artificial y las Redes Neuronales](#) - *Undeclared*
 - [2.1: Introducción General](#) - *Undeclared*
 - [2.2: Perspectiva Histórica](#) - *Undeclared*
 - [2.3: Aplicaciones presentes y futuras](#) - *Undeclared*
 - [2.4: Objetivos del Curso](#) - *Undeclared*
 - [3: Introducción al Cálculo Multivariante](#) - *Undeclared*
 - [4: Composición de Funciones. Regla de la Cadena](#) - *Undeclared*
 - [5: Aproximación Lineal de una Función Escalar de Varias Variables](#) - *Undeclared*
 - [6: Optimización y Método del Gradiente Descendiente](#) - *Undeclared*
 - [7: Redes Neuronales](#) - *Undeclared*
 - [7.1: ¿Qué es una red neuronal?](#) - *Undeclared*
 - [7.2: Perceptrón](#) - *Undeclared*
 - [7.3: Funciones de Activación](#) - *Undeclared*
 - [7.4: Funciones de Pérdida](#) - *Undeclared*
 - [7.5: Perceptrón Multicapa](#) - *Undeclared*
 - [7.6: Ejemplo- Regresión lineal](#) - *Undeclared*
 - [8: Tipos de redes neuronales](#) - *Undeclared*
 - [8.1: Redes neuronales superficiales](#) - *Undeclared*
 - [8.2: Redes neuronales profundas](#) - *Undeclared*
 - [9: Retropropagación](#) - *Undeclared*
 - [10: Entrenamiento de una Red Neuronal](#) - *Undeclared*
 - [11: Apéndices](#) - *Undeclared*
 - [11.1: Índice alfabético](#) - *Undeclared*
 - [11.2: Glosario](#) - *Undeclared*
 - [11.3: Detailed Licensing](#) - *Undeclared*
 - [11.4: Bibliografía Recomendada](#) - *Undeclared*

11.3: Detailed Licensing is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.4: Bibliografía Recomendada

Mathematics for Machine Learning

- <https://mml-book.github.io/>
- PDF <https://mml-book.github.io/book/mml-book.pdf>

Dive into Deep Learning

- https://d2l.ai/chapter_appendix-mathematics-for-deep-learning/index.html
- PDF <https://d2l.ai/d2l-en.pdf>

Coursera:

- <https://www.coursera.org/learn/multivariate-calculus-machine-learning>

Curso de Introducción a la Inteligencia Artificial:

- <https://www.elementsofai.com/>
- <https://buildingai.elementsofai.com>

Blogs:

- <http://www.aprendemachinelearning.com/>
- <https://towardsdatascience.com/three-month-plan-to-learn-mathematics-behind-machine-learning-74335a578740>

11.4: Bibliografía Recomendada is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.1: Índice alfabético

C

composicion de funciones

[4: Composición de Funciones. Regla de la Cadena](#)

L

Loss Functions

[7.4: Funciones de Pérdida](#)

R

regla de la cadena

[4: Composición de Funciones. Regla de la Cadena](#)

11.1: Índice alfabético is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.2: Glosario

Sample Word 1 | Sample Definition 1

11.2: Glosario is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.3: Detailed Licensing

Overview

Title: [Las matemáticas de la inteligencia artificial](#)

Webpages: 32

All licenses found:

- [Undeclared](#): 100% (32 pages)

By Page

- [Las matemáticas de la inteligencia artificial](#) - *Undeclared*
 - [1: Portada](#) - *Undeclared*
 - [1.1: Titulo](#) - *Undeclared*
 - [1.2: Página de Información](#) - *Undeclared*
 - [1.3: Table of Contents](#) - *Undeclared*
 - [1.4: Licensing](#) - *Undeclared*
 - [2: Introducción a la Inteligencia Artificial y las Redes Neuronales](#) - *Undeclared*
 - [2.1: Introducción General](#) - *Undeclared*
 - [2.2: Perspectiva Histórica](#) - *Undeclared*
 - [2.3: Aplicaciones presentes y futuras](#) - *Undeclared*
 - [2.4: Objetivos del Curso](#) - *Undeclared*
 - [3: Introducción al Cálculo Multivariante](#) - *Undeclared*
 - [4: Composición de Funciones. Regla de la Cadena](#) - *Undeclared*
 - [5: Aproximación Lineal de una Función Escalar de Varias Variables](#) - *Undeclared*
 - [6: Optimización y Método del Gradiente Descendiente](#) - *Undeclared*
 - [7: Redes Neuronales](#) - *Undeclared*
 - [7.1: ¿Qué es una red neuronal?](#) - *Undeclared*
 - [7.2: Perceptrón](#) - *Undeclared*
 - [7.3: Funciones de Activación](#) - *Undeclared*
 - [7.4: Funciones de Pérdida](#) - *Undeclared*
 - [7.5: Perceptrón Multicapa](#) - *Undeclared*
 - [7.6: Ejemplo- Regresión lineal](#) - *Undeclared*
 - [8: Tipos de redes neuronales](#) - *Undeclared*
 - [8.1: Redes neuronales superficiales](#) - *Undeclared*
 - [8.2: Redes neuronales profundas](#) - *Undeclared*
 - [9: Retropropagación](#) - *Undeclared*
 - [10: Entrenamiento de una Red Neuronal](#) - *Undeclared*
 - [11: Apéndices](#) - *Undeclared*
 - [11.1: Índice alfabético](#) - *Undeclared*
 - [11.2: Glosario](#) - *Undeclared*
 - [11.3: Detailed Licensing](#) - *Undeclared*
 - [11.4: Bibliografía Recomendada](#) - *Undeclared*

11.3: Detailed Licensing is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.4: Bibliografía Recomendada

Mathematics for Machine Learning

- <https://mml-book.github.io/>
- PDF <https://mml-book.github.io/book/mml-book.pdf>

Dive into Deep Learning

- https://d2l.ai/chapter_appendix-mathematics-for-deep-learning/index.html
- PDF <https://d2l.ai/d2l-en.pdf>

Coursera:

- <https://www.coursera.org/learn/multivariate-calculus-machine-learning>

Curso de Introducción a la Inteligencia Artificial:

- <https://www.elementsofai.com/>
- <https://buildingai.elementsofai.com>

Blogs:

- <http://www.aprendemachinelearning.com/>
- <https://towardsdatascience.com/three-month-plan-to-learn-mathematics-behind-machine-learning-74335a578740>

11.4: Bibliografía Recomendada is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.